Checking or Recomputing? Two Cost Regimes in Verifying AI-Generated Formal Artifacts

Anonymous research draft

Abstract

The cost of checking released AI-generated mathematics depends on what its proof asks the machine to do. A mathematical witness can be checked without repeating its producer's search, while a computational certificate can require the kernel to recompute the calculation encoded in the proof. We characterize these two cost regimes using released `openai/math` artifacts and their version-matched Mathlib dependency. [**Measured: E0231**] The same comment/string-masked scanner finds 1,354,379 `decide +kernel` sites in 163,097 generated-library files, versus 7 sites in 8,529 Mathlib files: approximately 10,668.3 times the source-megabyte density. Only 5,848 generated-library files (3.5856%) contain this form. This is lexical concentration, not a measured fraction of verification time. [**Partial: runtime weights incomplete**] Runtime-weighted prevalence remains [**RESULT: pending from the incomplete native sample**]. We examine repeated imports and production/checking coupling, and test single-process streaming and source-qualified finite-index chunking as design-implication prototypes. [**Measured: E0235**] Four selected matched closure checks cost 6.76–14.55% of serial source compiler work. These aggregates hide costly recomputation within computational targets. [**Measured: E0235**] A dependency-aware prototype reduces independent replay wall time 3.33-fold on one compiled module, with 27.8% more native CPU. The meaning of *verified* is a separate boundary: the default build omits 33.30% of kernel-decision sites, outcomes depend on settings and observation, and whole-library axiom closure remains unestablished. [**Measured: E0254**] At one source-compilation budget point, predicted ordering completes no more modules than FIFO; upper admission reduces interruptions and completions. The contribution is a characterization and its tested implications, not a general accelerator or a replacement trusted kernel.

1 Introduction

Checking a proof need not resemble producing it. Mathematical proof production can involve search, tactics and elaboration; an independent kernel can instead check the supplied witness. A computational proof exposes a different boundary: when its justification depends on reducing a large decidable expression, checking executes the relevant computation. Replaying the unchanged certificate can repeat that work.

Our primary workload is the released `openai/math` research library [33]. Its pinned Mathlib dependency supplies a version-matched reference [12]. The generated release contains far more source-level kernel evaluation, concentrated in a small subset of files. This motivates a distinction between mathematical-witness checking and proof-embedded recomputation, rather than treating verification as one uniform solver call. Files can mix both regimes; tokens cannot assign CPU.

The distinction suggests two systems responses. For mathematical proofs, can checking avoid producer elaboration and repeated import preparation? For computation, can independent claims be checked by several unchanged kernels and assembled into the original theorem? A faster untrusted numeric answer, weaker import trust scope or new axiom is not equivalent verification. We organize the study around four questions:

RQ1 Where do verification costs fall, and how prevalent are the computation-replaying and mathematical-proof-checking regimes across the library?

RQ2 Why does independent verification fail to realize the potential low cost of mathematical-proof checking?

RQ3 Can computational checking scale across cores or a cluster without expanding the trusted base?

RQ4 What gaps remain between a reported verified result and build coverage, reproducible outcomes and known axiom dependencies?

Claim scope after the prior-work audit. We distinguish *Measured*, a version/input/clock-qualified observation with a linked experiment; *Partial*, an observation missing stated evidence for the broader claim; and *Unproven*, a candidate without a qualifying result. These labels assess evidence, not novelty or general soundness. An established technique can produce a new empirical observation.

A: cost prediction and admission. [Partial: E0254; bounded trials, no per-instance guarantee.] Prediction-based resource decisions already have algorithm-selection and proof-task scheduling precedents [48, 6]. Our source-only forecasts have substantial

held-out errors on the completed subset, and one native budgeted ordering test completes no more modules than FIFO. Upper admission avoids interruptions with fewer completions. A separate known-root artifact trial improves admission availability but fails unseen-root transfer. These are limits and scoped tradeoffs, not a new predictor or safe resource bound.

B: measured boundaries of reproducible work accounting. [Measured: E0261; E0262; pinned proof and source scopes.] A later three-module census of2,524 eligible stored computational proofs has zero observed allocation-count spread across the tested thread counts. Two mathematical source programs have observed spans of0.0367% and0.1385%; a separate concurrent candidate panel has a maximum theorem-command span of2.33%, without acceptance-set changes. These are empirical applicability limits and observed count allowances, rather than a newly proposed deterministic-budget mechanism. **[Unproven: required acceptance-preserving budget margins and general replay.]** Lean heartbeats and SMT-LIB reproducible limits already replace elapsed time with internal work accounting [24, 3]. Applying and testing that idea here is not a new deterministic-budget mechanism. An upstream shared-thunk fix already addresses scheduling-dependent heartbeat charging [17]; its backport status in our pinned SDK is not established. A deterministic local counter does not establish a deterministic whole concurrent pipeline or turn exhaustion into mathematical rejection.

C: parallel decomposition and checked composition. [Partial: selected cases measured; general automation and full costs missing.] Proof futures, asynchronous checking and certificate decomposition predate this study [46, 2, 42]. We apply them to source-qualified finite-index certificates and compare chunking with declaration parallelism (Table 19). This is a bounded engineering case, not the first parallel checker or a general automatic splitter. Additional coordinator/adapter trust means RQ3's stronger unchanged-trusted-base goal remains open.

Each RQ separates evidence, missing results and design implications. We use public generated-software outputs and retained candidate streams as contrasts, never generate replacements for unavailable releases. Streaming and chunking are prototypes testing the characterization; warm imports, ordinary scheduling and external search/checking are prior techniques. Missing native profiles and full-library weights remain explicit.

2 Methodology

2.1 Sources and the common lexical boundary

The primary release is `4eaee87548250b2682affda7d05d947c05554bae`, with Lean 4.34.1 and Mathlib `d13f23b723b8a846827a245b89c10fc7d3f11612`. We compare all released Lean files in the primary inventory with the reference repository's `Mathlib/` subtree, excluding tests, archive and tooling directories from that reference denominator. The reference source tree is checked at its exact Git revision. All 163,097 primary source members are independently rescanned from the pinned official archive and matched by path/byte size to the complete inventory; decoding and lexical checks succeed. The local extracted cache is partial and is not used as the population. A complete received-archive-prefix scan and full-archive scan agree exactly; they are repeated validation, not additional corpus observations. The same existing scanner erases nested comments and string literals and matches `decide`, whitespace, `+`, whitespace and `kernel`, allowing zero whitespace. One MB is 1,000,000 original UTF-8 source bytes. Sites and containing files are not dynamic calls, distinct propositions, accepted proofs or native time.

Vericoding releases 1,503 model-tagged Lean snapshots, 1,490 distinct source texts; its original 626 are a subset [45]. Publisher labels are not our native verdicts or axiom audit. The inspected Vero release does not establish availability of evaluated agent outputs [50]; scaffolds/references do not replace them. Lean Kernel Verify supplies checking code and KIR fixtures with separate output provenance [30]. **[RESULT: Version-qualified generated-software native costs and axiom audit.]** The contrast streams retain 7,808 miniF2F, 5,952 ProofNet per recorded pass and 3,566 AutoVerus sources, including unsuccessful attempts. Source-sorted replay is not historical arrival traffic.

2.2 Cost boundaries and regime classification

Source production includes imports, elaboration, tactics, proof construction, first checking and requested artifact emission. AI inference time is excluded unless separately supplied by publisher logs. Independent checking consumes compiled declarations under a declared trust scope. A kernel-only subclock omits preparation/cleanup and cannot replace either complete process clock.

The regimes describe checking work, not disjoint proof languages. Computational checking normalizes embedded calculations; mathematical checking can avoid the producer's search and elaboration. A module may contain both. A C++ checker sample includes inference,

equality, normalization and cleanup, not solely final validation. User-cycle percentages are not elapsed-time shares. **[RESULT: Representative no-memory-option source-compilation phase profiles, classification error, observer uncertainty and runtime-weighted prevalence.]**

2.3 Sampling and current native point

The original frame remains 163,097 modules, with 21 nonempty strata formed from four source-size bins, three internal-import-depth bins and masked kernel-decision presence. Seed 20261030 selects two draws per stratum, 42 total. Inclusion probability is $2/N_h$; purposeful modules and prerequisites receive no additional population weight. For complete strata,

$$\widehat{T}_h = N_h \bar{y}_h, \qquad \widehat{V}_h = N_h^2 (1 - n_h/N_h) s_h^2 / n_h, \quad n_h = 2.$$

Missing draws stay missing, not replaced by completed-response counts. Two-draw normal-cost intervals are fragile under heavy tails. The retained W no-`--M` source panel initially covers 29 original random draws and two purposeful modules. A completed 48-process continuation adds 46 prerequisites and two original draws, not 48 new probability samples. All original sources, no-`-M` arguments, PID/birth retirement and matching native clock sums are checked. A later 19-job continuation contributes one more original draw, LowBlock15_1; its other original draw is uninvoked after a prerequisite timeout. **[Measured: E0276]** Source-frame coverage is now 38/42 with 19 complete strata and 4 missing costs. Target-only paired replay still covers 31 draws/14 strata, and the dynamic checking-grain panel retains its separate denominator. Prerequisites and purposeful modules receive no sampling weight. These are local compilation costs with prerequisite objects available, not cold transitive builds, whole Lake execution or generation time. **[RESULT: Complete no-memory-option sample and per-regime cost estimates.]**

The platform is phobos-vt, Intel Core i9-14900K, observed 64,205,640 KiB RAM. Timed native captures share one P0–7 lock; preparation/analysis use E cores. Eight requested workers are not assumed to use eight CPU cores. Current measurements omit `-M`, including `-MO`. The required memory-protection boundary is external cgroup; a result records its actual enforcement rather than assuming conformance. The W point uses 15 GiB charged native memory and separate one-second sampled native/controller RSS response. Charged pages, leader RSS and tree RSS differ; sampled response can overshoot and is not a kernel hard aggregate-RSS quota. Every result must record its actual enforcement boundary.

A measurement correction, not a workload finding. The pinned runtime initializes its native memory limit to zero; `check_memory` returns before RSS queries while zero [22]. Our earlier optional `-M` setting introduced periodic accounting. Three ordered plain repetitions per arm give guarded/no-`--M` wall-median ratios 2.03/1.61/1.01 for Grid/UMoments9/EnvelopePsiB, with matching verdicts, diagnostics and emitted artifacts. The same outer 15 GiB cgroup remains; output and heartbeat settings match within each module. These passing cases are not resource-failure equivalence. The earlier five-source `openai/math` panel's 75–94% versus roughly 2% user-cycle contrast, and a 13-short-module 1.28 replay/build ratio, also used `-M` builds. The panel is not a Mathlib native sample. They motivate questions, not current runtime conclusions. Affected costs/projections remain archived, never scaled by Grid's ratio.

2.4 Outcomes, observation and trust

Process wall, user/system CPU, queue/capture lifecycle and extra rechecks keep their own denominators. Unprofiled, sampled and traced executions are separate conditions. Ordinary error, timeout, cgroup OOM, watchdog interruption, platform loss and never-started dependency are distinct; none proves falsehood. Raw, failed and partial captures remain retained on the PVC.

Import-relative stock `leanchecker` trusts imported declarations. Its `--fresh` mode replays the loaded constant map into an empty kernel. Both use the upstream kernel and designated unsafe/partial exclusions [25]; neither is an independent implementation or an axiom-closure audit. A cheaper trust scope cannot substitute for equivalence.

3 Kernel-evaluation forms are concentrated; runtime weights remain open

RQ1: checking versus recomputing. [Partial: E0231; global runtime weights missing.] Table 1 gives the complete, common-boundary source comparison. Unmasked Mathlib text contains eight sites in five files. One is a documentation comment in `Mathlib/Tactic/Echelon/Cert.lean`. Removing comments gives seven in four files; the table uses this common masked boundary, not eight unmasked sites against masked generated code.

The density ratio is 10,668.3, approximately 1.1×10^4. Only 3.5856% of generated-library files contain the form. These are concrete candidates for inspecting recomputation, not an exhaustive runtime classification: expensive reduction can occur without the token, and a positive file can mix computation and symbolic reasoning. The 38 completed no-`--M` random source targets sum to 554.806 seconds of native wall intervals and 867.533 user-plus-system CPU seconds. This is not a

Corpus	Lean files	Source MB	Kernel-decision sites	Containing files	Sites/MB
Mathlib `d13f`	8,529	98.450	7	4	0.0711
`openai/math`	163,097	1785.499	1,354,379	5,848	758.544

Table 1: [**Measured: E0231**] Nested-comment/string-masked `decide +kernel` sites. Decimal MB uses original source bytes. Lexical counts are not time weights.

full-build makespan or population expansion. The separate purposeful Grid and Amplitudes points differ in output/heartbeat scope and do not fill missing strata. Unprofiled process costs do not supply a kernel-cycle partition. A source-defined proxy marks files containing a masked explicit kernel-decision form; it does not establish which dynamic phase dominates them. [**Measured: E0276; one further original-frame source cost.**] TrackedTestBounds completes in 5.199 native elapsed seconds and 5.412 CPU seconds. Its untimed E-core prerequisites receive no sampling weight. It completes the matching 9,146-module ordinary stratum, increasing the complete-stratum domain to 161,034 modules across 19 strata. Stratified compiler CPU is 57.546 hours in marked files and 189.317 hours in other files, a conditional marked share of 23.31%. All nine marked strata have both draws, covering 5,848 marked modules; two ordinary strata and 2,063 ordinary modules remain incompletely observed. These are source-compilation estimates, including imports, tactics and emission, not kernel-check-only hours or semantic computation-regime weights. The earlier 14-, 15-, 16- and 17-stratum shares, 17.14%, 33.02%, 26.47% and 26.37%, use different covered domains, rather than repeated measurements of the same population. Changes reflect newly observed cost and coverage, not a temporal trend for a fixed population. Two draws per stratum and ordinary-class nonresponse still leave full-library CPU and the global marked-time share unestablished. [**RESULT: Representative no-`-M` phase profiles and full-library regime-specific cost weights after completing the original sample.**] The 3.5856% source-file concentration must not become a cost fraction.

Clock choice and nonresponse alter the estimand. [**Measured: E0276; same 19 complete source strata, offline arithmetic.**] The conditional marked share is 23.31% of weighted compiler CPU and 18.59% of summed native elapsed time, a 4.72-percentage-point difference. The latter adds native process durations; it is not parallel Lake makespan or an isolated checking-phase share of whole-library time. CPU weights include worker execution; elapsed weights reflect each module's wall duration at the measured concurrency point. Earlier clock contrasts use different covered domains and are not a population time trend.

The two incomplete strata contain 2,063 of 163,097 modules (1.26%), not necessarily that many unbuilt files. Four original source costs are absent. The 38 known terms in the eventual complete-sample Horvitz–Thompson point sum to 246.863 CPU-hours, equal here to the complete-stratum subtotal because both incomplete strata have zero observed draws. This is not a lower confidence limit or a zero-filled population estimate. Increasing every missing draw by one CPU second would increase the eventual point by 0.573 CPU-hours under the fixed original weights. This sensitivity does not predict missing costs or variance.

Narrow missing-cost scenarios do not imply precise sampling. [**Partial: E0277; nominal sampling uncertainty for the 19 complete strata.**] Recomputing the finite-population stratified variance from the same two original draws per stratum gives an approximate 95% Student-t interval of 43.99–449.74 CPU-hours around the covered-domain estimate of 246.863 hours, with Satterthwaite effective degrees of freedom 2.13. Two strata contribute 62.41% and 27.61% of the estimated variance. This nominal interval is fragile with two observations per cell; it omits the two incomplete strata, nonresponse, runtime drift and protocol-transfer uncertainty. It is neither a full-frame confidence interval nor a guaranteed bound. The narrower missing-cost scenarios below address a different uncertainty and do not establish population-estimate precision.

Whole-frame costs require explicit missing-cost scenarios. [**Partial: E0276; fixed 38-source sensitivity, not measurements or confidence limits.**] Preserving 38 known costs and original inclusion weights, assume the four remaining ordinary-marker draws cost between the observed ordinary minimum and maximum, 1.204525–40.791652 CPU-s. Whole-frame construction scenarios then span 247.5–270.3 CPU-hours, with endpoints rounded outward. The earlier 37-draw 242.2–315.2-hour range remains a dated assumption scenario; the smaller current width follows an additional observed cost.

A separate transfer model gives 16.8–24.0 checking CPU-hours. It assigns ordinary modules the complete checkpoint-closure CPU fractions 0.063124–0.089514 and marked modules the older unsegmented computation fractions 0.084421–0.084671. This deliberately mixes protocol-specific anchors; it is neither a measured matched protocol nor evidence that source markers identify checking cost. The earlier 16.5–28.0-hour transfer used the 37-cost snapshot. Neither model is measured whole-library checking cost, a confidence interval

CPU forecast	Median error	P90 error
Constant	86.50%	366.74%
Source bytes	34.03%	100.55%
Source/imports	48.08%	77.62%
Static source features	34.95%	77.38%
Static plus marker regime	50.07%	93.02%

Table 2: [**Measured: E0254**] Absolute relative errors on the 31 completed original draws, project-grouped five-fold validation. These are compiler CPU forecasts, not independent-kernel costs, population estimates or error bounds.

or a guaranteed saving. Their known terms use all 38 observed draws; they also differ from RQ2's trusted-import target checks and complete selected closures.

The LowBlock15_1 observation remains 184.589 CPU-s, above the old 31-draw marked-class maximum of 163.846 CPU-s, demonstrating that observed support is only an assumption. Without missing-cost assumptions, the completion-cost upper remains unbounded. Assigning all missing draws ten times the observed ordinary maximum raises the source scenario to 480.6 CPU-hours. At the current class-maximum source scenario, giving missing draws no checking savings yields 40.2 modeled checking CPU-hours; twice their source work yields 63.6 and four times yields 110.3, below the 270.2-hour source value. A separate stress assumption of twelve times their source work yields 297.3 checking CPU-hours, reversing that comparison. These are different stress assumptions, not measured checking costs. A subunit factor assigned to unknown modules cannot establish general benefit. All four unobserved original source costs remain missing in the primary evidence.

Source features do not establish bounded prediction error. [**Measured: E0254; 31 completed source-compiler observations.**] Five fixed log-ridge models predict native compiler CPU from source-only features, with no Lean execution or post-production proof/artifact features. Their penalty, transforms, marker-based domains and splits are frozen before fitting. Project folds hold whole source projects out; ordinary module cross-validation can leak shared project structure. Table 2 shows project-held-out errors. Adding explicit computation markers and their interactions does not consistently improve prediction; paired project-bootstrap intervals for the error differences include zero. This does not establish a harmful effect either, or refute the distinction between checking work regimes.

The eleven costs absent from that forecast capture are not imputed into its 31-input evaluation. An auxiliary set has 6,214 compiler modules and 5,085 conditional replay-wall labels, mostly dependencies; it does not make the primary sample complete or turn compiler CPU into kernel replay cost. The separate calibration-fit upper estimates cover 28 of 31 observed inputs. Small dependent projects and nonresponse prevent guaranteed future coverage, let alone a deterministic per-request bound. Confidence intervals resample project blocks of fixed held-out errors, rather than the complete training procedure. [**Partial: completed-subset forecasts; missing/censored heavy inputs and general calibration remain unresolved.**]

A source label can match observed kernel work without bounding cost. [**Measured: E0256; independent reconstruction of retained call traces.**] The fixed explicit-marker rule agrees with a narrower dynamic reference on 27 of 29 observed original draws. The reference is whether decision-shaped helpers consume at least half of that module's measured kernel-call CPU, not all possible large reductions or production-cost dominance. The confusion counts are 14 true positives, 13 true negatives, one false positive and one false negative. Both errors belong to the Catalan project: one marked module has 43.79% helper CPU, while an unmarked ordinary `decide` elaborates to helpers using 61.67%. Expensive unrecognized proof forms can escape this definition.

A conditional project-block bootstrap spans 77.78–100% accuracy over 20 observed projects. Thirteen original dynamic draws remain absent: even unweighted accuracy on the 42-draw frame can range from 27/42 to 40/42 without assumptions about them. The primary panel deliberately oversamples marker-positive sources; 93.10% is not population accuracy. The two extra purposeful hot modules both match, but stay outside that rate. Agreement with W's same source rule is scanner consistency, not independent dynamic evidence.

Mislabeled modules contain 1.0284 of 213.3623 kernel-call CPU seconds (0.482%). This is exposed work, not additional CPU caused by choosing a wrong checker: that penalty needs both routes on those inputs. Classification, numeric forecasting and safe admission are separate obligations. Cost-sensitive algorithm selection without runtime regression is itself established prior work [49]. [**Partial: operational label only; missing-label, population and routing-penalty evidence remain unresolved.**]

Native checking work has different useful granularities. [**Measured: E0234; conditional replay panel.**] The earlier checking-grain capture is a different panel, unchanged by the additional source-compilation observations. It replays 31 retained compiled targets: 29 available probability-sample artifacts and two purposeful modules. It contains 34 successful native target executions, including stock/timed pairs for UMoments9, BarrierData41 and Grid. Each target trusts its declared imported environment and replays its own eligible declarations; this is not full-closure checking or source compilation. A wrapper around the original C++ checking call records per-call thread CPU. Decision-shaped

Module	Calls	CPU (s)	Max (ms)	≥ 1s
UMoments9	1164	75.436	85.5	0
BarrierData41	1320	46.910	164.6	0
Grid	4608	263.619	185.7	0
EnvelopePsiB	40	58.006	6163.3	10
ResiduesModThree	2	3.620	2274.5	2

Table 3: [**Measured: E0234**] Decision-shaped theorem checking in five selected modules from the complete 31-artifact replay panel. Calls, CPU and maxima refer to these declarations, not all module work; ≥ 1s uses thread CPU.

theorem bodies have `of_decide_eq_true` at their head after annotation cleanup. This syntactic proof-term class does not exhaust computational proofs; it is defined by stored terms rather than the source tactic name `native_decide`.

Grid's 4,608 such declarations cost 263.619 CPU seconds in aggregate, but none exceeds 186 ms individually (Table 3). UMoments9 and BarrierData41 similarly contain many subsecond calls. EnvelopePsiB instead has ten calls above one CPU second, totaling 55.782 of its 58.006 seconds (96.17%). These require different decomposition opportunities despite both carrying kernel-decision forms.

Across the unweighted panel, all native checking calls consume 588.773 thread-CPU seconds; this decision-shaped class consumes 574.744 seconds. Its twelve calls above one second consume 59.402 seconds: 10.09% of all checking-call CPU, or 10.34% of this class's CPU. Weighting only the 29 available probability-sample artifacts gives 28.23% and 34.93%, respectively. These are conditional available-case ratios, not unbiased full-library estimates or upper bounds on parallelizable work: thirteen original draws are absent and nonresponse is not corrected. They must not be multiplied into source-compiler or whole-closure costs.

The stock/timed acceptance sets agree for all three paired modules. Their single ordered timed/stock replay-CPU ratios are 0.9100/1.0016/1.0745. Formatting occurs outside each native-call timer, but allocation/cache and observer effects can still change the experiment. This is not a zero-overhead or unperturbed source-profile claim. The capture records P0–7, no `-M`, an external 14 GiB charged-memory cap and a separate sampled RSS response. All 34 target processes and their cgroups, plus the controller, are retired.

Checking an unchanged computation-based certificate repeats the reductions needed by its justification. It need not repeat the producer's search, but cannot treat an untrusted numerical answer as a theorem. For a mathematical witness a low checking share is possible when source production dominates; it is not established for every mathematical module.

System-design implication. [Partial: implication of scoped observations; general benefit unproven.] Choose the acceptance boundary before the accelerator. Mathematical witnesses motivate production/checking separation; computational certificates motivate independent kernel work. Measure the native grain before dividing it: many short declarations suggest dependency-aware declaration parallelism, while measured long calls can justify within-goal splitting. Tokens and old guarded profiles establish neither a global partition nor an accelerator ceiling.

4 Cheap checking can be hidden by imports and production coupling

RQ2: why independent verification remains costly. [Partial: selected costs measured; causal import decomposition missing.] Source compilation imports, elaborates, executes tactics and constructs proofs before checking. A post-compilation checker can omit that production, but a new process still locates objects and prepares its environment. A potentially small kernel cost therefore need not imply cheap independent verification.

The earlier no-`-M` stock-checker point completes 15 target replays. For 13 selected short modules its process-wall median is 3.510 seconds, including lookup, imports, replay and cleanup. It is not an exclusive kernel timer or a contemporaneous construction pair. It cannot establish a current construction/replay ratio or its cause by division against an earlier, differently configured build. **[RESULT: Matched no-`-M` import-only/checking/cleanup components and a comparable independent-process baseline decomposition.]** Repeated imports and coupling are explanations to test, not a residual assigned to an unmeasured stage.

Trust scope changes the work too. Separate no-`-M` controls take 2.195 seconds import-relative and 73.134 seconds fresh-scope for Mathlib Nat.Basic, a 33.32 ratio. A NearPinDeletion fresh-scope control reaches its 180-second wall bound. These are scope contrasts, not interchangeable verification methods. A stream that checks dependencies needs a full-closure baseline.

Fresh target replay saves work within a narrower trust scope. [Measured: E0260; 31 matched original-draw targets.] All 31 fresh target-replay processes accept the retained target declarations; their target artifact parts match the measured producer copies byte-for-byte. Each process loads trusted compiled imports before replaying the target, as in the module-local checker boundary. Full transitive artifact-byte identity is unestablished, and the earlier namespace-setup failures remain archived. These results are target checks,

Observed clock (s)	Source	Target replay	Ratio
Native CPU sum	632.590	272.695	0.4311
Native elapsed sum	393.896	274.209	0.6961

Table 4: Thirty-one completed original-draw source/target pairs. Compiled imports remain trusted in target replay. Neither ratio is whole-library verification, a parallel makespan or a production speedup.

not independent closure checks.

Every observed module has a replay/source ratio below one in both clocks. The median module CPU ratio is0.451 (observed range0.087–0.885), distinct from the ratio of sums in Table 4. Source and replay are ordered observations, without randomized ordering or repeated pairs. The queue-inclusive controller clock is14,768.395s, separate from the 274.209s sum of native intervals. CPU excludes coordination and lock waiting. The no-`-M`, P0–7, requested-eight-thread processes have a15GiB charged cap and a separate sampled-RSS response; all recorded identities and cgroups retire without resource stops.

[Partial: conditional paired estimator; full-frame cost and context matching remain missing.] For the original 31-draw replay panel's 14 complete paired strata, original inverse probabilities give 140.852 source CPU-hours versus54.906 target-replay CPU-hours, a ratio of0.3898. The nominal paired t/delta95% interval is[0,0.9401]; with two draws per stratum and effective degrees of freedom about1.19, it is fragile and excludes nonresponse, machine drift and context uncertainty. The ordinary and marked subgroup intervals each extend above one. Eleven original source/paired draws remain missing, so this is not a full-library cost estimate. These target-only costs cannot replace the empty-environment closure-stream costs below, nor can those selected-closure ratios replace this paired target estimand.

Prototype: streamed original-kernel replay in one process. [Measured: E0235; four selected overlapping closures.] O's implementation reads module-data parts, traverses imports, retains their mapped regions and calls the upstream `Kernel.Environment.replay`. A visited set avoids processing an already visited module within the stream. Checking compiled declarations is separate from source elaboration. The implementation also has an import-relative single-module mode and malformed-proof controls. The first complete matched control covers the dependency closure of Mathlib `Data.Nat.Basic`: 1,330 modules and 132,484 declaration entries. Every module is reconstructed with the stock Lean 4.34.1 compiler, requesting `.olean`, `.ilean` and C output. The streamed checker then consumes these newly emitted objects, rather than the initially distributed cache, from one empty checked kernel environment. All commands

Closure (modules)	Source (s)	Check (s)	Ratio
`Nat.Basic` (1,330)	622.379	65.693	10.56%
NearPin (3,333)	2556.623	172.811	6.76%
Grid (3,793)	3878.707	563.082	14.52%
Amplitudes (5,083)	6173.195	898.034	14.55%

Table 5: **[Measured: E0235]** One-pass no-`-M`, same-artifact closure comparisons. Source work sums compiler subprocess wall; checking is one stream's process wall. Shared dependencies are counted once within each closure, so rows are not independent trials. These are different tasks, not parallel Lake or serving speedups. Computational closures exceed the listed O-line share.

omit `-M` and use P0–7 with external cgroup protection.

The checking process takes 65.693 seconds versus 622.379 seconds of serial source compiler work (Table 5). This is a comparison between producing and validating artifacts, not a 9.47-fold acceleration of production or serving. The numerator includes process setup, object reads, replay and retirement; the denominator includes each module's imports, elaboration, tactics, first checking and standard artifact emission. Neither clock includes shared-lock wait or coordinator work. The streamed process consumes 65.509 user and 0.163 system CPU seconds; its `wait4` peak RSS is 754,252 KiB.

Forty-two completed single-module-oracle chunks independently reimport each target's environment before native replay; every chunk exits zero and its requested and accepted sets agree. Together they match all 1,330 streamed acceptances. A separate stock-checker preflight covers 13 modules. Three invalid proof terms—a loose variable, unknown constant and wrong sort—are rejected through both streaming and module-local replay. These checks establish differential evidence for the tested artifacts, not universal transparency or an axiom-closure audit. Upstream replay excludes 2,269 unsafe/partial entries in this closure; axioms keep their ordinary Lean semantics. The stock reader's well-formed-object assumption is retained.

The closure includes compiler and standard libraries, not only mathematical theorems. Its 80 Mathlib-namespace modules account for 0.849 seconds of kernel replay and 49.607 seconds of source compiler work. That component ratio (1.71%) omits shared-base checking, reads and process overhead; this namespace also contains tactics and helpers. It cannot replace the full-closure 10.56% process comparison or establish a mathematical-module population estimate. This one-pass, warm-cache control demonstrates a lower-cost checking boundary at complete dependency scope. It does not isolate the causal shares of import reuse and omitted production.

A computational target and its dependencies have different ratios. [Measured: E0235; target and closure clocks differ.] The reconstructed

Target	Source (s)	Component (s)	Ratio
`Nat.Basic`	0.410718	0.026844	6.54%
NearPinDeletion	0.962274	0.004473	0.46%
Grid	318.231	255.489	80.28%
Amplitudes	133.615	110.026	82.35%

Table 6: [**Measured: E0235**] Matched target read/map/kernel-replay components after dependency checking, versus target source compiler process wall. Components exclude shared-base checking and startup; these are not standalone latency ratios.

Grid closure also completes: 3,793 source modules, 3,878.707 seconds of serial compiler work, and 563.082 seconds of matched stream checking (14.52%). Its exact accepted set matches 119 completed module-oracle chunks, including 42 previously completed chunks for identical shared dependencies. The stream reports 382,707 entries and 3,554 upstream unsafe/partial exclusions. This advances the earlier observation whose module-local parity was pending. The predeclared Amplitudes and `NearPinDeletion` cases now also complete both construction and exact module-oracle acceptance parity, with 5,083 and 3,333 modules, respectively. Their matched stream/source-work fractions are 14.55% and 6.76%. All four cases use one 5,085-node source union: shared immutable dependencies are built once and charged to each containing closure. There are 161 completed oracle chunks covering those 5,085 unique modules, not four disjoint samples or repeated confidence trials.

Within that process, Grid's own kernel replay takes 255.487 seconds, versus 318.231 seconds for its source compiler invocation: 80.28%. The target replay subclock excludes shared-process startup and dependency checking; source compilation includes target imports and artifact emission. This component ratio is not standalone checking latency or a sampled CPU share. Table 6 uses a common read/map/replay subclock for the four targets. The selected AI mathematical lemma `NearPinDeletion` needs 4.47 ms at that boundary versus 0.962 seconds of source compiler wall; Amplitudes needs 110.026 versus 133.615 seconds. The contrast supports two different cost regimes on these inputs, without assigning every remaining producer millisecond to imports. The smaller whole-closure fraction includes source work for dependencies and cannot establish that a computational target itself is cheap to validate. No source-only or timing classifier for every module follows.

Closure totals can hide the cost of generated nodes. [**Measured: E0246; selected provenance composition.**] We also split each selected closure by source provenance, rather than assigning an entire closure to its root's proof style. The four Grid `OAI` modules account for 9.76% of its summed source compiler work

Closure	OAI modules	Source share	Replay share
NearPinDeletion	2	0.0743%	0.00627%
Grid	4	9.76%	52.70%
Amplitudes	8	8.69%	46.93%

Table 7: [**Measured: E0246**] Shares attributable to fixed-source `OAI` modules within each complete selected closure. Source shares use summed native compiler wall; replay shares use summed per-module monotonic replay-call intervals after dependencies have been checked. Replay shares exclude read/map, startup, logging and cleanup, and are not full-process CPU shares. These are overlapping selected closures, not population-weighted samples.

but 52.70% of its measured replay-call intervals (Table 7). The corresponding eight Amplitudes modules account for 8.69% and 46.93%. Thus removing source production changes which part of the dependency graph accounts for the work; a low complete-closure check/source ratio can coexist with substantial computational replay in generated nodes.

The generated modules' replay intervals total 294.675 seconds for Grid and 421.071 seconds for Amplitudes, versus 378.547 and 536.442 seconds of their respective source compiler work: 77.84% and 78.49%. By contrast, the two generated NearPinDeletion-closure modules require 10.80 ms of replay versus 1.898 seconds of compiler work, or 0.569%. The 1,330-module Mathlib control contains no `OAI` nodes. Membership is established by namespace and pinned source path, not a complete semantic classifier of computational proofs. The two clocks measure different tasks: compilation includes imports, elaboration, tactics and artifact emission, whereas replay consumes compiled declarations in a checked environment. These shares do not isolate a causal import cost, establish a standalone latency gain, or fill the missing whole-library regime weights. Shared dependencies make the closures correlated; all values come from the earlier single-pass captures, not new repetitions.

The actual O native cgroup limit is 11.875 GiB. Grid source compilation reaches 7,940,196 KiB peak RSS (7.57 GiB), exceeding the listed 4 GiB O-line share; the streamed closure reaches 2,328,540 KiB. This is an observed resource deviation, not a qualified 4 GiB comparison or authority to increase that share. [**RESULT: Repeated generated-library closure comparisons, causal component attribution, uncertainty and full axiom coverage.**] The complete passing-set comparisons do not establish universal shared-context transparency: duplicate theorem names, conflicting definitions and undeclared sibling visibility require separate qualification. A context-auditing continuation is not part of these timing results.

Policy	Done	CPU (s)	Cuts	Deferred
FIFO	15	59.44	1	15
Predicted order	15	59.46	1	15
Upper admission	13	41.85	0	18

Table 8: [**Measured: E0254**] One fixed-order native 60-CPU-second budget per policy. Completion counts are not a production throughput gain. A cut is an operating-system budget interruption, not a mathematical rejection.

Policy	Done	Cuts	Discarded	Unused
FIFO	0.33	1.00	97.34	0.65
Type route	0.33	1.33	97.33	0.61
Upper admission	1.00	0.00	0.00	54.05

Table 9: [**Measured: E0256**] Means across three cyclic arrivals per policy, each with a 120-CPU-second budget. Done counts whole roots; discarded and unused CPU are seconds. These are descriptive observations, not three same-arrival repetitions or a population confidence interval.

Predictive admission trades completion for fewer budget cuts. [**Measured: E0254; one ordered source-compilation policy point.**] A separate native test gives FIFO, predicted-cost ordering and upper admission 60 CPU seconds each, compiling the same 31 eligible original source modules one child at a time. These are distinct module proofs, not alternative candidates for one theorem. Frozen held-out forecasts are precomputed; extraction and forecasting cost are excluded. Each policy charges child and driver CPU. The test uses P0–7, eight compiler threads, no `-M`, normal `.olean`/`.ilean`/C outputs, and retained filesystem caches. Approximate operating-system CPU limits are not Lean heartbeat semantics.

All 43 completed calls exit zero; two other calls receive `SIGXCPU`. Upper admission leaves 18.15 allocated CPU seconds unused and completes fewer modules. Different selected IDs and ordered warm runs prevent a causal latency comparison from their wall clocks. A matched offline replay at the same budget and arrival order also yields 15/15/13 completions. A separate larger-budget, randomized-arrival simulation favors predicted ordering, but is neither this native test nor evidence of a simulator/native contradiction. No general pass-at-k follows from one reference proof per distinct module.

The native child cap is 11.5 GiB, with a 512 MiB parent allowance, exceeding the listed O-line 4 GiB share; this is not retroactively qualified as a 4 GiB experiment. Maximum `wait4` RSS is 6.76 GiB while the charged peak is 937,037,824 bytes, measured at different accounting boundaries. All 45 recorded temporary cgroups are absent after completion. Per-child PID/birth records are not emitted by this runner; complete `wait4` outputs establish returns, not an invented independent identity audit. There is one observed policy order, not replicated service-level benefit or a bounded-error admission mechanism.

A correct regime label is insufficient for CPU-budgeted routing. [**Measured: E0256; separate compiled-artifact admission experiment.**] A second native point uses three selected root requests: Nat.Basic, BarrierData41 and Grid, each including its complete dependency closure. A fixed source label routes the ordinary root to serial checking and the two marked roots to the qualified four-worker original-kernel checker, without proof rewriting. FIFO uses serial checking for all three. Upper admission adds historical type envelopes from these same roots; this is recurrent known-work calibration, not a bound learned from class alone.

Nine batches rotate three cyclic arrivals and policy order, allocating 120 CPU seconds per policy in each arrival. Queue wait, source production, offline features and plan generation are excluded. Only whole completed roots count; partial dependency accepts do not complete a request.

Type routing completes no more roots than FIFO, although mean batch wall is 76.87 rather than 119.49 seconds. Spending the same work sooner before interruption is not higher successful-completion throughput. All five completed native requests are the ordinary root, with exactly the reference's 1,330 accepted IDs; seven calls receive `SIGXCPU`. Neither computational root completes at this budget. Their prior full-pair parity is not newly established by interrupted requests. One routed batch dispatches a second interrupted request because discrete CPU limits leave a whole-second allowance; this quantization is retained. No observed batch exceeds its recorded CPU allocation.

Upper admission finishes the ordinary root in every arrival and avoids discarded-root work, but defers both computational roots and leaves 45.04% of allocated CPU unused. The existing ordinary NearPin closure takes 172.73 CPU seconds, already exceeding the frozen ordinary envelope of 71.74. Class membership therefore fails as an unseen-work upper bound. Completed parallel full-pair costs also consume more CPU than serial; a label's wall-time opportunity need not improve a CPU-budget objective. The three selected requests have one reference each, not a pass-at-k study or a representative service mix.

Commands retain no `-M`, P0–7 and the shared lock. All recorded native identities and child cgroups retire, with no OOM, sampled-RSS stop or wall timeout. The existing O child cap remains 11.5 GiB with a 512 MiB driver allowance, above its listed 4 GiB share. Peak sampled child RSS is 1,389,350,912 bytes and charged memory is 322,310,144 bytes; the driver peak is unrecorded. This does not establish whole-lane 4 GiB compliance.

Closure / checking protocol	Check / source (s)	Ratio
Falconer, checkpoints	2,043.59 / 22,077.77	9.26%
Mathlib, historical stream	65.69 / 622.38	10.56%
Mathlib, checkpoints	91.28 / 622.38	14.67%

Table 10: [**Measured: E0265**] Three mathematical-closure wall-time observations. Falconer has 13,205 modules/27 processes; both Mathlib Nat.Basic rows have 1,330 modules, with one and three processes respectively. Each ratio uses its own complete checking/source wall sums.

[**Partial: known-panel availability benefit; unseen admission guarantees, representative mixes and end-to-end feature cost missing.**]

A large project completes through checked-prefix recovery. [**Measured: E0264; one complete segmented capture.**] The Falconer extension contains 13,205 modules: 2,461 generated modules and 10,744 dependencies. All have successful source-compiler records, including 5,077 exact common artifacts reused from the same capture and charged once to this closure. Their summed native source-compiler CPU is 32,292.604s. A chain of 27 successful checker processes accepts the complete module set using 2,038.444 CPU-s, or 6.31% of source-compiler CPU. Here and in the matched Mathlib control, CPU means summed native user plus system time, with checking and source work charged over their respective complete selected closures. The separate native elapsed sums are 2,043.585s for checking and 22,077.770s for source compilation, yielding a wall-time ratio of 9.26%. All native module names and import edges match the planned closure. The final event records 800,218 input entries, 4,451 upstream unsafe/partial exclusions and 19 module-local fallbacks. Sixteen new module-local reference chunks cover 8,128 modules; the remaining 5,077 retain exact-artifact qualification from the original capture. The resulting reference and segmented acceptance sets both contain 13,205 IDs. This is finite positive-set agreement, not universal context or axiom closure.

[**Measured: E0268; retained raw process costs, no new native run.**] Table 11 compares native CPU under the same checkpoint adapter and 500-new-module quota: 6.31% for Falconer and 8.95% for Mathlib. Both selected mathematical closures use substantially less CPU for checking than for source production. Source CPU includes imports, elaboration, tactics, initial checking and artifact emission; it is not an isolated kernel baseline.

Table 10 separately retains the wall-time observations: 9.26% for Falconer, 10.56% for the historical Mathlib stream and 14.67% for checkpointed Mathlib (91.279s versus 622.379s). CPU and wall ratios answer different questions and are not pooled. Each closure has one ordered capture with retained page cache. The older unsegmented Mathlib result uses another adapter; it is not a causal control for the checkpoint protocol. Neither the CPU comparison nor the wall comparison isolates AI authorship, tactics, dependency composition or import cost, and neither estimates population cost or production latency.

Closure	Check / source CPU (s)	Ratio
Falconer	2,038.44 / 32,292.60	6.31%
Mathlib Nat.Basic	91.15 / 1,018.22	8.95%

Table 11: [**Measured: E0268**] Summed native user-plus-system CPU for complete selected closures under the same checkpoint protocol. This clock differs from the wall-time sums in Table 10.

Each segment exports original-kernel-accepted declarations and audit metadata; the next stock importer trusts that certified delta before checking new terms. Startup, artifact reads, all 26 snapshot exports and 26 prefix restores, and native exit are within the 2,043.59s summed process clock. Snapshot write-return costs follow normal page-cache semantics; isolated snapshot seconds and physical-flush durability latency were not measured. Queue wait, driver reduction, reference qualification and producer/tool preparation are excluded. The clock ends immediately after wait4, before collector/cgroup cleanup. Five finite checkpoint cases span 18 native processes, including two bad-proof rejections in restore sequences. Serialized audit metadata and the adapter remain trusted implementation code; arbitrary supplied snapshots are outside this protocol. Frozen-input continuation is not a cross-candidate cache or an uninterrupted single-process check.

[**Partial: resource and generalization boundaries remain.**] The earlier full attempts both end at their 1,500s limits: 8,942 and 9,040 complete accept records, with unterminated output fragments. Their 3,000.416s of failed native work stay separate from the successful segmented cost. Neither timeout is a mathematical rejection or a complete-set mismatch. The source cells use 11.875GiB (reused) or 11.75GiB (new) charged caps; the new checker uses 11.75GiB and reaches 10.368GiB wait4 RSS. These exceed the listed O 4GiB share and are not a qualified 4GiB comparison. Independent sampled RSS and exact native-child PID/birth records were not emitted by this collector; absent cgroups and retired recorded controllers do not establish a complete process-tree audit. [**RESULT: Repeated complete-project captures, causal component attribution and whole axiom/context qualification.**]

A bounded whole-source attempt remains partial. [**Partial: E0271; incomplete source coverage and scoped producer diagnostics.**] A separate finite-window attempt accepts only 2,488 of the

163,097 source modules, plus 10,744 dependency modules; 160,609 source modules remain without accepted checking records. Its terminal state is incomplete. The production ledger records 9,124 source-compilation returns of `rc=1`. [**Measured: E0271; retained diagnostic census and finite controls.**] Of these, 9,118 report unavailable interpreter or native declarations, four lack module data and two report IR/code-generation errors. The diagnostics do not establish a single failing phase for all invocations. A separate four-case qualification, using complete original package paths, compiles three pinned source files named in the failures and an Init import control; an earlier metadata-only copy retry still fails. These controls also differ in output facets and resource limits, so they neither isolate one missing runtime component nor provide a timing comparison. They do not establish that every failed source will compile after repair; the two IR/code-generation cases still need qualified retries. None of the 9,124 returns is counted as a qualified kernel or mathematical rejection, and the four controls do not increase source acceptance coverage. The accepted prefix does not establish whole-library checking cost, validate the RQ1 scenario ranges, or replace the complete selected Falconer closure. No missing module is assigned zero cost.

A stopped certificate project yields a limited module-local sample. [**Measured: E0275; complete 20-file sample, incomplete directory.**] Ramsey production stops after 190 of 444 target files, consuming 22,146.098 native CPU-s for those new source files; no full-closure stream runs. Four files are sampled without replacement from each of five equal-sized source-byte strata of the retained 190-file prefix (seed 20261103). All 20 separate original-kernel module-local processes accept. Their summed checking CPU is 815.679s versus 2,271.357s for the same files' source compilation, a pooled ratio of 35.91%. Individual CPU ratios have P10/median/P90 of 35.51/40.62/48.83%. Startup and imported-context loading are charged to each separate checker process, and those imports remain trusted. These observations therefore do not form a matched comparison with the shared-stream closure ratios above.

Weighting each sampled stratum mean by its 38 retained files estimates 7,748.952 CPU-s for independent module-local checking of the 190-file prefix, or 34.99% of its known source CPU. A within-stratum percentile bootstrap (10,000 resamples) gives 29.41–40.41%; with only four observations per stratum, this describes resampling variation inside the observed prefix, not uncertainty from dependency-order selection or the 254 unbuilt targets. The built files all belong to Construction; their median source size is 4,457 bytes versus 81,652.5 bytes among the unbuilt files. Only 24 of 180 kernel-marked files are built. Neither the sample fraction nor its interval is extrapolated to all 444 files or to whole-closure streaming.

A separate, excluded-from-sampling reference checks CertificatesK10 in 356.443 CPU-s against 1,725.134 source CPU-s (20.66%). This finite-certificate file has no `+kernel` marker but uses ordinary `by decide`; marker absence cannot identify a cheap mathematical case. The small-file process/import floor and the large certificate work must remain distinct from a pure kernel-phase fraction. [**Partial: trusted imports, one observation per sampled file, a selected prefix, and the recorded O resource-share deviation remain; full-directory cost, axiom closure and repeated captures are missing.**]

System-design implication. [**Partial: implication of scoped observations; general benefit unproven.**] Expose the cheap checking boundary if one exists: separate production and checking APIs, reuse compatible contexts and charge the complete lifecycle. Warm imports are established engineering [34, 36]; the characterization must show where they matter and what remains expensive.

5 Recomputation motivates parallel original-kernel checking

RQ3: scale without adding trusted proof rules. [**Partial: finite transformations; unchanged implementation TCB unproven.**] A monolithic reduction can leave cores unavailable to one checking invocation. Parallelism requires independently checkable claims and a checked connection to the unchanged original proposition. A source split, numeric evaluator or worker success flag alone is insufficient.

The existing module-parallel baseline has a different boundary. [**Measured: E0247; source audit only, no throughput comparison.**] The bundled checker already launches `replayFromImports` tasks for selected modules before collecting their results [20]. Each task loads its imports through the stock importer and checks its local declarations; task dispatch does not wait for imported modules' checking tasks to finish. Our shared stream instead reuses a checked prefix. A dependency-DAG model that requires every imported module to finish checking before the next module starts therefore adds a barrier absent from the existing task dispatcher. Such a model is not an upper bound on the bundled checker. Module-local tasks also repeat environment preparation, so shared-stream call weights alone cannot predict their process costs. The CLI selects module-name prefixes, which need not equal an explicitly chosen artifact closure. [**RESULT: Native same-closure module-local task comparison with matched workers, memory protection, acceptance scope and complete lifecycle accounting.**] The stream speedups below compare the stated

Arm	Kernel span (ms)	Worker CPU (ms)	Native (s)
Original	146.08	146.03	1.974
$k = 1$	144.07	144.04	5.973
$k = 2$	140.50	149.35	5.862
$k = 4$	151.68	237.61	5.897
$k = 8$	178.02	392.06	5.927

Table 12: [**Measured: E0234**] Medians of three no-`-M` repetitions on one real Grid equality. Kernel span covers the worker checks; worker CPU sums their thread clocks. Native wall includes preparation, checks and artifact handling.

serial baselines; they do not establish superiority over this unmeasured native module-parallel path.

Prototype: generated claims and checked assembly. [Measured: E0234; small Grid whole-path negative.] M freezes a real Grid `PointBox.calc` equality and structurally forms 18 scalar-equality leaves. The prototype groups them for one, two, four or eight workers, constructs decision proofs and submits declarations through the original C++ kernel's `Environment.addDeclCore` API. Checked leaves are emitted as modules; a composition proof uses extensionality lemmas and is checked against the original target. Code checks the target, sorry/metavariable absence and expected axiom boundary. The completed flat-partition diagnostic below exercises this acceptance path.

The controls distinguish original monolithic $k = 0$, the same decomposition with $k = 1$, and $k = 2, 4, 8$. This separates decomposition from parallel execution. Preparation, repeated reductions, serialization/communication, assembly, checking, memory and retirement belong in end-to-end cost. The corrected flat-partition matrix completes 15 native cells: the original and $k = 1, 2, 4, 8$, each repeated three times. All accept; final propositions and the `[propext]` axiom dependencies match the extracted original, with no `sorry` or new axiom dependency. Both original and split paths export a checked final module. Same-k repeated artifacts are byte-identical; different partitions intentionally construct different proof terms.

Table 12 shows a negative whole-path result for this partition. Relative to the same decomposition at $k = 1$, pure-kernel scaling is 1.025, 0.950 and 0.809 at $k = 2, 4, 8$; the eight-job case spends 2.72 times the worker CPU. More occupied cores do not imply useful acceleration. The split native pipeline takes roughly three times the original's wall time. At $k = 2, 4, 8$, ordinary module reimport takes median 3.643/3.659/3.660 seconds; final theorem checking takes about 0.16–0.20 ms. Environment reconstruction, rather than the small assembly proof alone, dominates the observed merge stage. This standalone goal pipeline is not a warmed service or a full Grid build.

The flat grouping places both exponential record components in one $k = 2$ job, while several $k = 8$ jobs touch the same components. This supports an imbalance/repeated-reduction hypothesis, not a complete CPU attribution. A subsequent component-preserving control also completes 15 accepting cells. Its original/k = $1/k = 4$ kernel-span medians are 151.03/147.72/78.75 ms, showing a local reduction at this boundary. Whole native wall nevertheless rises from 1.982 seconds original to 5.838 seconds at $k = 4$. Twenty-seven same-k artifact byte comparisons differ; native inspection finds exact typed `ConstantInfo` equality for all 27 pairs, including theorem types and bodies. This is not extension-state equality or an established explanation of serialization differences. Both controls remain negative for end-to-end acceleration of this small goal, not bounds on other decompositions or larger goals.

Both partition points' recorded PID/birth identities are retired. Their recorded resource response is a 14 GiB sampled tree-RSS stop with a 180-second self alarm. Their frozen runners and receipts do not establish a dedicated charged-memory cgroup, so these diagnostics are not presented as the required final cgroup-controlled resource comparison. No observed cell times out or reaches its sampled stop. Aggregate heartbeat equivalence to the monolithic request is not implemented. [**RESULT: Final resource-qualified parallel comparison, corrupt-leaf and budget/failure equivalence, shape coverage and heterogeneous-module transfer.**] One frozen equality is not full Grid or a general extraction algorithm.

Declaration-level replay on a complete module. The measured subsecond calls motivate a second prototype: dependency-aware replay of the original declarations, without rewriting their proof terms. Barrier-Data41 has 3,425 supported safe, acyclic declarations. The prototype creates ready frontiers, checks jobs with the stock kernel, exports checked module fragments and normally imports them for successor frontiers. It explicitly rejects unsupported or cyclic input. The final native audit compares every typed `ConstantInfo`, including theorem bodies, and returns names in the original array order.

All 15 cells accept the same 3,425 typed constants and ordered names; all 99 native stages exit zero and their recorded identities/cgroups retire. Eight workers reduce the measured pipeline median from 46.992 to 14.127 seconds, 3.326-fold against the direct stock-API baseline (Table 13); per-repetition ratios range from 3.284 to 3.359. Native CPU increases 27.8%. One-worker wave replay is 13.5% slower than direct stock replay, so a wave-one-only baseline would overstate the gain. The point uses P0–7, no `-M`, and an external 14 GiB charged-memory cap.

The output contract is the final typed-constant/name audit. Intermediate serialization is a cost of this implementation, not an extra required output imposed on

Replay path	Pipeline wall (s)	Native CPU (s)
Direct stock API	46.992	46.956
Wave, 1 worker	53.321	53.205
Wave, 2 workers	31.072	54.556
Wave, 4 workers	20.233	58.335
Wave, 8 workers	14.127	60.016

Table 13: [**Measured: E0235; scoped prototype.**] BarrierData41 independent compiled-module replay, three ordered repetitions per path. Wall spans first stage-configuration write through final audit retirement, including graph planning, native processes, imports, exports and driver gaps. Lease wait and initial directory/environment setup are excluded; CPU sums native processes, excluding the Python coordinator.

Path	Kernel span (s)	Runtime (s)	CPU (s)
Original	6.125	9.848	9.836
1 chunk	6.177	14.347	14.319
2 chunks	3.531	11.490	14.892
4 chunks	1.967	10.331	15.307
8 chunks	0.973	8.982	15.110

Table 14: [**Measured: E0236**] EnvelopePsiB helper, medians of three repetitions. Kernel span runs from the first worker's stock checking-call entry to the last exit. Runtime includes both sequential producer and merger process lifecycles, imports, exports and interphase cleanup. Native CPU excludes the Python parent; transformation generation, base/merge-lemma compilation and the original library producer build are excluded.

the direct baseline. Original proof terms and the original kernel are retained under this local checked-fragment protocol; this is not a proof for arbitrary coordinators, untrusted object transfer or changed failure/budget semantics. The import-relative dependency trust matches the stock module API. Upstream checking already dispatches independent modules as tasks [20]; this case isolates parallelism inside one module, without establishing whole-library or source-elaboration speedup. Other modules, warm-service throughput and repeated failure behavior remain [**RESULT: unmeasured at the matched resource/trust boundary**].

Parallel leaves can lose their gain at the module boundary. A larger measured helper, `envelopeExpB16_calc` in EnvelopePsiB, checks 31 finite-index interval-exponential equalities. The prepared transformation groups indices into 1, 2, 4 or 8 chunks, retaining each index's original computation. An original-kernel-checked merge lemma reconstructs the original target; an explicit equality control connects that target to the released helper. This case tests parallel rechecking of a prepared transformation, not the cost of automatically producing that transformation.

Eight chunks shorten the kernel span 6.292-fold, but runtime decreases only from 9.848 to 8.982 seconds: 1.096-fold, or 8.79% less wall time. Native CPU rises 53.6%. Paired runtime ratios are 1.096, 1.103 and 0.949; the last repetition is 5.35% slower than its original. Three repetitions, with reversed setting order in the second, do not establish population uncertainty or guaranteed nonregression. At eight chunks, merger import alone takes a median 3.142 seconds, whereas final kernel assembly takes 0.296 ms. The paired nonkernel residual added over the original is 4.24–4.91 seconds; this residual includes multiple operations and is not a causal attribution to imports alone.

All 15 performance cells accept the same extracted target and the exact axiom set `[propext]`, without `sorry` or added axioms. A false-decision control reaches explicit original-kernel rejection. The 28 recorded native phases retire with their cgroups, with P0–7, no `-M` and an external 14 GiB charged-memory cap. Sampled own-tree RSS peaks at 9.087 GB and charged memory at 2.588 GB; these are distinct measurements, not equivalent limits. All 84 same-setting repeat artifact comparisons are recorded equal; proof artifacts intentionally differ across chunk counts. Aggregate heartbeat-budget transparency and arbitrary tactic-backtracking parity remain unimplemented. This helper's small and variable runtime gain complements the Barrier module case; their different baseline and output interfaces prevent combining the speedup ratios into a library estimate. It also identifies a concrete design cost: preserve checking parallelism without rebuilding a prepared environment at every assembly boundary.

Checking inside one prepared stream. A combined prototype avoids module-export round trips between target trials. It checks the import prefix once, then compares direct stock replay with 1, 2, 4 and 8-worker declaration replay against the same immutable checked prefix. Unlike the earlier checked-fragment protocol, it commits checked results through an internal-ABI adapter; its additional implementation trust is discussed in Section 6.

One completed repaired v14 Grid process checks its 3,793-module closure and runs five ordered target trials (Table 15). Each parallel trial processes 7,898 declarations in eight dependency waves; its complete typed constants, including proof bodies, match direct serial replay. There is no chunk substitution in these Grid trials.

Eight workers reduce the incremental target wall time 4.024-fold, with 97.76% more process CPU. CPU/wall is 7.95, an observed average occupancy rather than a count of operating-system threads. Four workers take 94.183 seconds; one worker remains slower than direct serial replay. The complete process takes 1,054.529 seconds and 2,116.247 cgroup CPU seconds, including the prefix and all five trials. Neither that total nor the incremental ratios measure a complete single-mode speedup.

Grid trial	Incremental wall (s)	Process CPU (s)
Direct serial	265.010	264.853
1 worker	270.264	270.158
2 workers	155.577	310.739
4 workers	94.183	375.643
8 workers	65.856	523.784

Table 15: [**Measured: E0244**] One ordered Grid trial sequence inside one process and checked prefix. Trial clocks include planning, checking and the result-identity audit; they exclude the already checked prefix and process startup. These are incremental target costs, not complete independent invocations. There is one observation per path, without randomized order or confidence bounds.

Stream path	Wall median [min,max] (s)	CPU (s)
Original serial	68.605 [65.989,71.805]	68.570
Audited serial	71.326 [71.304,72.125]	71.180
1 worker	77.859 [76.171,77.967]	77.438
2 workers	57.569 [57.529,58.216]	80.811
4 workers	47.598 [47.362,49.214]	84.644
8 workers	45.050 [44.270,45.292]	93.430

Table 16: [**Measured: E0241**] Complete artifact-closure process costs on repaired frozen v14, three fresh-process repetitions per path in repeated fixed order. Filesystem caches are not flushed. CPU is a separate median. The fresh original-stream baseline is distinct from the earlier single 65.693-second observation and from source compiler work.

The process exits successfully under P0–7/no-`-M`/ 14 GiB charged-memory protection; sampled RSS and charged peaks are 2.936 GB and 0.919 GB, respectively. Upstream unsafe/partial exclusions remain 3,554 among 382,707 input entries. This repairs the earlier scheduling-defective requested-eight observation; the two versions are not pooled. One ordered observation does not establish repeated Grid uncertainty or library-wide performance, and the extra CPU has no matched causal profile.

A repaired complete-closure comparison. The later frozen version adds a shared worker-start gate and set-based import-visibility membership, while retaining the local-cycle repair. Its startup and audit costs remain inside the measured process boundary; we do not isolate either implementation change's causal benefit. A fresh matrix uses the same constructed 1,330-module `Mathlib.Data.Nat.Basic` closure for the original serial stream, audited serial replay and 1, 2, 4 or 8-worker streams (Table 16).

All 18 cells exit zero and accept the same 1,330-module set, with 132,484 entries and 2,269 upstream unsafe/partial exclusions per cell. The parallel paths execute 55,255 ordinary-declaration jobs in 3,616 waves across 858 modules; 472 complex/cyclic modules use upstream serial replay. There are no module-local context fallbacks on this fixed closure, and certificate proof rewriting is inactive.

Eight workers reduce process-wall median 34.33%, or 1.523-fold, with 36.25% more native CPU. Same-repeat wall ratios are 1.515, 1.594 and 1.491. Audited and one-worker paths remain slower than the original stream. Three fixed-order repetitions do not establish population confidence or production-service throughput. All native identities, temporary cgroups and the finite matrix controller retire under P0–7/no-`-M`/14 GiB charged protection. The repaired source requalifies 20 native boundary controls and the three actual-certificate scenarios. This completes one repaired-version artifact-checking comparison; it neither retroactively qualifies the defective v4 table nor proves universal context equivalence or an unchanged implementation TCB. The earlier 622.379-second source-work denominator is not remeasured here, and no compiler-build speedup follows from these same-artifact stream ratios.

A second complete process uses the same repaired frozen source for BarrierData41. It checks a 4,448-module prefix once and then runs the same five ordered target paths. Incremental serial and 1/2/4/8-worker wall costs are 53.960, 56.443, 29.829, 16.468 and 10.186 seconds. All parallel trials return the same 3,425 typed constants, including proof bodies, across six dependency waves. Eight workers reduce target wall time 5.297-fold, with process CPU rising from 53.934 to 80.155 seconds, or 48.62%. CPU/wall at that boundary is about 7.87, without implying only eight operating-system threads. The complete process takes 468.112 seconds and 867.564 CPU seconds, including the checked prefix and all five target trials. It accepts 4,449 modules with 3,595 upstream unsafe/partial exclusions; all recorded native identities and cgroups retire without resource stops. This is one ordered observation per path, not repeated uncertainty. Its incremental 5.297-fold result is distinct from the earlier 3.326-fold checked-fragment pipeline result: neither their baselines nor their lifecycle/output boundaries match. We do not attribute the difference to the worker-start gate or report it as a cold-mode, source-build or library-wide improvement.

Complete closure costs are smaller gains than target curves. [**Measured: E0244; one ordered pair per closure.**] Two new pairs run the original serial stream and repaired uniform four-worker stream from empty checked environments on the same artifacts (Table 17). Each arm uses one complete configuration, independently checking every dependency, with retained filesystem caches. The external charged-memory cap is 14 GiB, both arms have an 800-second native budget, and all timed native execution uses P0–7 without `-M`. No certificate proof rewriting is active.

Barrier's complete closure takes 35.80% less wall with

Closure	Modules	Serial (s)	Four (s)	Ratio
BarrierData41	4,449	474.851	304.862	1.558
Grid	3,793	557.568	296.293	1.882

Table 17: [**Measured: E0244**] Actual complete native-process pairs, one observation per arm. Clocks run from subprocess launch through `wait4`; subsequent sampler join and cgroup removal are verified but outside that wall clock. Each pair runs serial first. Grid's arms are separated by a controller repair gap; its completed reference is retained without rerunning.

24.63% more cgroup CPU (474.582 to 591.467 seconds). Grid takes 46.86% less wall with 33.56% more cgroup CPU (557.350 to 744.406 seconds). Complete `wait4` CPU accounting is retained separately and differs from cgroup usage by less than 0.001 seconds for each of these arms. The full accepted sets and import graphs match between each pair and its corresponding earlier complete-prefix curve: 425,463 entries with 3,595 upstream exclusions for Barrier, and 382,707 with 3,554 for Grid. All sampled native identities, controllers and temporary cgroups retire without resource stops or OOM events.

The first Grid controller finishes its serial arm and then fails while writing pair metadata, before launching the second arm. The corrected continuation reuses that result and measures only the missing parallel arm. We compare the two native-process clocks, not a continuous pair-session time across the repair gap. Neither pair is randomized or repeated. Grid's earlier matching source work is 3,878.707 seconds, a serial sum of compiler invocations rather than a parallel Lake build wall time; the measured stream pair does not remeasure that production cost. A matching complete source-work sum for Barrier is [**RESULT: Barrier closure source work**]; its leaf compiler time cannot fill that cell. The complete four-worker ratios cannot be replaced by, or multiplied with, the eight-worker incremental target ratios. These pairs do not measure complete eight-worker costs for that version or general service throughput. A later-version heterogeneous closure curve is reported separately below. The coordinator and internal-ABI adapter still add trusted implementation code; acceptance-set parity is not a proof of universal malformed-input equivalence.

More heterogeneous workers do not accelerate the full closure. [Measured: E0266; one fixed-order observation per setting.] A later frozen driver checks the same complete 4,449-module Barrier closure from an empty checked environment at 8, 12, 16 and 24 requested workers (Table 18). The allowed CPU masks expand from eight performance cores (8P) to 8P+4E, 8P+8E and 8P+16E, where E denotes efficiency cores. This changes both worker count and available hardware; it is not homogeneous strong scaling or a matched comparison with the earlier v14 four-worker result. All four

Hardware / workers	Wall (s)	Native CPU (s)
8P / 8	284.720	668.786
8P+4E / 12	299.807	809.696
8P+8E / 16	310.591	937.934
8P+16E / 24	319.705	1172.473

Table 18: [**Measured: E0266**] Complete Barrier closure, one fresh process per setting in the fixed order shown, with retained filesystem caches. Wall spans launch through `wait4`, including reads, planning, checking and native result audit; sampler join and cgroup deletion follow the clock. CPU is native user plus system time. No per-target observer is enabled for this curve.

invocations retain the original kernel and proof terms, accept the same module set as the retained serial reference, and traverse the same import graph: 425,463 entries and 3,595 upstream unsafe/partial exclusions. The added coordinator and internal-ABI adapter remain trusted implementation.

At 24 versus eight workers, wall time increases 12.29% and native CPU increases 75.31%. Each setting routes 1,231 complex/cyclic modules through the upstream fallback and 3,218 through parallel replay. At 8P, fallback windows take 99.819 seconds and time outside recorded kernel-work windows takes 46.726 seconds, together 51.47% of process wall. The parallel-route windows also increase, from 138.175 to 161.525 seconds. These are disjoint elapsed windows, not a causal CPU profile; a fallback module can contain ordinary work, so its entire cost is not proved inherently serial. Workers may migrate within their allowed masks. Live snapshots show E-core placements in the 12- and 16-worker cells; the 24-worker mask is recorded at launch without a live thread snapshot. External E-core load, frequency, P/E-specific CPU shares and order effects are not independently controlled. The four processes and their cgroups retire without a sampled resource stop or OOM, under separate 14 GiB charged-memory and sampled-RSS protections and without `-M`. This observed negative result bounds this heterogeneous extension: the earlier P-core gains do not imply continued complete-closure speedup when adding E cores. It neither isolates contention or scheduling as the cause nor establishes a hardware-independent scaling limit, repeated uncertainty or cluster behavior. The separately instrumented resident-leaf observation in Section 6 uses different output and timing boundaries; its counts and leaf costs cannot replace these complete-process observations.

Observed CPU growth extends inside the checking calls. [Measured: E0273; a separate corrected placement and observer protocol.] Twelve fresh-process cells each check the same 4,449-module Barrier closure and match the retained serial reference's accepted IDs and import graph. The ear-

lier two-worker placement trial is excluded: two active workers had shared CPU0. The corrected driver gives active tasks distinct per-wave slots; its same-source profile-on P1/P2/P4/P8 wall times are 655.448/433.980/346.351/315.062 seconds, with native CPU 647.069/625.008/648.965/723.951 seconds. E16 and P-first mixed24 take 425.347/325.296 wall seconds and 1,121.948/1,100.116 CPU seconds. These are different versions and treatments from Table 18, not a continuous tuning curve. Observed use of up to 24 cores is reported without retroactively changing the earlier eight-core allocation.

Eight observer-enabled full cells have the same 237,863 ordinary checks and 3,283,925,639 observed allocation units, including identical per-module sums. At P4 to P8, native CPU rises 74.986 seconds, of which 71.953 seconds (95.95%) lie in the observer brackets around original-kernel calls. Those brackets include heartbeat and timing reads; they are not a pure kernel-cycle count. The observer records thread CPU, interval-union wall occupancy and per-check start cores; occupancy must not be added as concurrent CPU. Implementation-serialized preparation, complex replay and duplicate checks are not proved inherent dependency lower bounds.

Natural-placement P8 to mixed24 adds 19.230 wall seconds with the observer enabled: 4.629 seconds in implementation-serialized windows, 7.410 in ordinary-call wall occupancy, 3.386 in batch time outside that occupancy, and 3.805 in the remaining process residual. Native CPU increases 458.725 seconds, including 438.492 in ordinary-call brackets. The natural-placement wall regression is 6.36% with the observer and 5.53% without it. Under deliberate P-first placement, the corresponding wall differences are +3.25% and -0.37%. Each cell is one ordered observation with retained filesystem caches; the sign-sensitive control differences do not isolate observer overhead, thermal drift or placement as the cause. Unpinned per-core attribution uses check-start cores and detects differing endpoints, without reconstructing intervening migrations. A fixed intrinsic serial fraction, atomic-reference-count cause, cache cause and frequency cause remain unestablished. All cells omit `-M`; charged-memory and sampled-RSS guards are separate. The original mathematical kernel is retained, while the coordinator, observer and internal-ABI adapter remain trusted implementation.

Whole-command counters narrow the CPU-growth hypotheses. [Measured: E0277; a separate complete 16-process Barrier matrix.] Each of four worker settings (4, 8, 1, 2, in that order) has ordinary before/after, counter and sampled-recording processes. All 16 accept the same 4,449-module closure with identical per-module check and allocation-count vectors: 237,863 checks and 3,283,925,639 units. Across the four ordinary 4-to-8-worker before/after pairings, 91.70–99.15% of extra native CPU lies inside the observer brackets around original kernel calls. As above, these include heartbeat and timing reads and are not pure C++ kernel cycles. The earlier 95.95% observation belongs to its separate capture.

In the whole-command user-mode counter runs, 4-to-8-worker instructions increase only 0.0396%, while cycles increase 8.31% and aggregate IPC falls from 1.834 to 1.694. Retired locked loads change 0.117%, L3-miss loads increase 11.98%, modified-line forwarding 46.41%, and demand-RFO snoop-HITM responses 49.76%. These counts cover startup, coordination and checking together; they are not gated to the observer brackets. Locked-load counts are operations, and forwarding/RFO events do not identify reference counting uniquely. The observations support investigating coherence, ownership and cache costs under nearly fixed instruction work, without isolating one cause or a removable stall fraction. Fixed order, retained page caches and one counter run per setting leave order, frequency and instrumentation effects unresolved. Original sampled-worker READ metadata is incomplete for some settings; later counting-only repair points have separate denominators and are not used here to fill missing original attribution.

Pooled certificates pass a broader native qualification. [Measured: E0243; finite input/child controls, not general soundness.] A separate native control selects all ten measured Envelope helpers whose earlier checking calls exceed one CPU second. Before transformation, each input `ConstantInfo` must equal its prepared selector, including the original proof. The original kernel then checks certificate support, 80 pooled children and ten assembled parent theorems; their transitive axiom union is `[propext]`, without `sorry`. Corrupting an incoming original proof or a prepared child produces explicit original-kernel rejection in two additional controls. The control imports its prefix through the stock loader and rechecks the ready local declarations; it does not independently check the entire closure. Its three native processes retire without resource stops, but it supplies no paired performance result. The earlier v10 complete-prefix attempt remains an RSS interruption; passing these controls does not repair that historical result. The subsequent complete-prefix observation below uses a different frozen version and prefix-worker policy.

Splitting must beat declaration parallelism. [Measured: E0243; one target sequence, no full single-mode pair.] A repaired v14 Envelope process independently checks 10,751 dependency modules before nine ordered target trials. The prefix uses four requested workers; target paths request 1, 2, 4 or 8. This uses a fresh checking environment, not flushed filesystem caches. All 110 leaf declaration names, types and universe levels match serial. Ordinary paths retain their

Envelope target path	Wall (s)	Process CPU (s)
Serial	61.913	61.889
Declaration, 1 worker	61.470	61.444
Declaration, 2 workers	33.856	67.204
Declaration, 4 workers	22.515	76.542
Declaration, 8 workers	17.539	97.815
Chunk, 1 worker	61.852	61.815
Chunk, 2 workers	37.920	72.995
Chunk, 4 workers	20.768	80.182
Chunk, 8 workers	13.299	102.888

Table 19: [**Measured: E0243**] One ordered Envelope target sequence after independently checking the shared dependency prefix. Clocks include payload reading, support replay, children, merges, ordinary declarations and result audit. Source scanning, producer compilation and certificate generation are excluded. Each path has one observation, not a repeated median.

original proof bodies. Chunk paths deliberately replace ten eligible parent proofs after matching their complete incoming constants to the prepared references.

The stronger baseline is declaration-level parallelism, not just serial checking. Eight-worker chunking takes 13.299 seconds versus 17.539 seconds for eight-worker declaration replay: 1.319-fold, or 24.17% less wall, with 5.19% more CPU. Against serial it is 4.655-fold, with 66.25% more CPU. Chunking at two workers is slower than two-worker declaration replay. The four chunk settings check 150 child proofs and 40 parent merges; the transformed parents' transitive axiom union remains `[propext]`, without `sorry`. These are source-qualified finite-index certificates, not a general decomposition algorithm or an untrusted selector.

The complete process accepts 10,752 modules, with 785,706 input entries and 4,444 upstream unsafe/partial exclusions, and retires in 1,665.530 seconds with 2,772.428 cgroup CPU seconds. Its 331.132 seconds of target trials leave 1,334.398 seconds of nontrial residual, including prefix verification, reading, startup and cleanup. That residual is not an import-only clock, and the complete process runs all nine paths rather than one chosen configuration. A separate original-stream serial attempt reaches its 1,680-second native limit, returning timeout status 124 after 1,680.158 seconds. It retains 10,744 complete acceptance records from the 10,752-module graph, plus one truncated 222-byte record; there is no completion event. The eight modules without complete acceptance records are unobserved at this boundary, not eight mathematical rejections. Cgroup CPU is 1,679.367 seconds, sampled RSS peaks at 7.830 GB and charged memory at 1.836 GB; no memory stop or kernel OOM occurs. The native process, sampled children, controller and cgroup retire. This is a time-censored original-stream attempt, not a completed serial reference or an acceptance-set discrepancy on completed inputs. It neither supplies a full-closure speedup nor identifies which checking call consumed the remaining budget. The earlier 1,665.530-second process includes a parallel prefix and all nine target paths, so dividing these two process clocks is invalid.

The one-second RSS samples peak at 13.971 GiB, below the 14 GiB sampled stop; `wait4` nevertheless records 14.019 GiB between observations. Charged peak is 8,910,696,448 bytes, below the separate 14 GiB charged cap. No resource stop or OOM occurs, and the native peak remains below the 16 GiB lane allocation; no hard 14 GiB RSS guarantee follows. All native identities, the controller and cgroup retire. This one success differs from the earlier v10 failure in both source and prefix-worker policy; it does not isolate a memory fix or establish near-budget robustness. The additional WaveExpData ancestor takes 44.990 seconds at its prefix checking-route boundary, but has no per-goal cost qualification in this capture and is outside the earlier ten-helper panel. [**RESULT: Repeated and single-configuration complete-closure comparisons, broader repaired-version coverage and aggregate failure/budget transparency.**]

A cluster cannot silently trust remote object files or worker verdicts as new proof assertions. Specify what the acceptance authority checks, dependency identity, transfer and recovery. [**RESULT: Multi-machine protocol, native soundness/differential evidence and complete cluster costs.**] Single-host code and multicore timings are not distributed results.

Earlier certificate proofs establish selected feasibility; seven Int/list original/certificate pairs all accept, but currently matched transfer costs remain unestablished. Changing a certified algorithm and parallelizing a fixed decomposition are distinct. Reflection and proof parallelism are established approaches [11, 13, 10].

System-design implication. [**Partial: implication of scoped observations; general benefit unproven.**] Expose work at a kernel-checkable boundary; retain original checking rules and the required axiom closure. Keeping generation/scheduling outside the trusted decision is a design requirement, not a proved property of the combined in-memory prototype. Preserve shared subcomputations when choosing work units and account for environment reconstruction. Validate decomposition and parallelism separately, then check the complete assembled theorem at equal CPU/memory resources. Faster leaves or transformed subgoal success alone are insufficient.

6 Verified leaves build, outcome and axiom boundaries open

RQ4: what has actually been established? [**Measured: E0231; lexical/build coverage only.**] The

publisher's default root reaches 130,834 of 163,097 modules. The other 32,263 contain 451,008 of 1,354,379 masked kernel-decision sites, 33.30%. This is not an omitted CPU fraction or a statement that outside files fail. All 562 PlanarPacking and 466 Transonic modules are outside. A successful default build cannot establish all released-artifact coverage.

Source sites, declarations and axioms differ. [Partial: retained-object census; whole axiom closure missing.] In the retained 363-object census, 45,661 decision-shaped auxiliaries have 42,610 globally distinct expression-type keys. Source occurrences can expand into several helpers; imported objects use another boundary. Neither these keys nor Lean's branch-local auxiliary cache establishes safe cross-proof reuse [19, 21]. Zero direct native-evaluation tokens does not establish zero transitive axiom dependence. Vericoding source flags and publisher labels similarly do not establish native acceptance or known axiom closure. **[RESULT: Repository-wide declaration reachability, admissions and axiom closure.]**

A shared context can change which proof is checked. [Measured: E0237; a specific acceptance-contract mismatch.] A later combined streaming prototype exposes a concrete context failure. A valid earlier sibling provides `Shared.t : True`. A different incoming module imports only `Init.Prelude`; its raw declarations give `Shared.helper : True := Shared.t` and `Shared.t : True := Shared.helper`. These are deliberately cyclic proof data, not successfully compiled source. Version 4 accepts the incoming module after the sibling: the old `Shared.t` makes the helper check, masking the incoming local cycle. The same binary's isolated module-local original-kernel path rejects it with an unknown-constant error for `Shared.t`.

This is a demonstrated acceptance-contract mismatch, not an original-kernel bug or an accepted proof of `False`: both claimed propositions are `True`. The defect is substituting an out-of-import binding while validating the incoming proof. The repaired prototype follows incoming local dependencies transitively; a cycle takes module-local replay. It rejects this input with one and eight workers, and completes 20 native boundary controls, including sibling visibility, corrupted duplicate proofs, conflicting definitions and stale/duplicate checked tokens. All 22 recorded control processes, including historical acceptance and the isolated rejection, retire without resource interruption. These controls bound the tested cases, not arbitrary corrupted modules. The previous 18-cell, 1,330-module positive-input performance table belongs to the defective version and is not a repaired-version throughput result. **[RESULT: Repaired-version performance beyond the selected closure and broader module-context differential evidence.]**

A second boundary is implementation trust. This combined in-memory prototype commits checked results through a checked-token coordinator and a pinned internal-ABI C map-insertion adapter. Those components are additional trusted implementation code, although the original kernel still checks mathematical declarations. They differ from the earlier Barrier checked-fragment/normal-import protocol. Neither prototype establishes a universally unchanged implementation TCB.

Measured kernel recomputation needs no extra count allowance. [Measured: E0265; the qualified three-module census.] Across all 2,524 eligible computational targets, allocation-heartbeat variation is 0% for every target under 1/2/4/8 workers and three repeats: 30,288 accepted checks, with P95=0% and maximum=0%. All 22,716 matched comparisons preserve counts, verdicts and diagnostics; there is no nonzero case in this census. For these measured computation-stage units, determinism therefore needs no extra allocation-count budget to cover the observed worker/repeat envelope. This is the bounded empirical meaning of determinism without an extra budget here, not a guarantee for future runs, all computational proofs or zero runtime overhead.

The mathematical source panels instead have an observed maximum span of 2.333868% (about 2.33%) over the separate 16-source concurrent-candidate panel, with 83/192 changed theorem-command counts (84/192 full command vectors) and no acceptance changes. The two full-source modules have spans 0.036739% and0.138453%, a coarse 0.03–0.14% module-level envelope, with identical judgments. These different units and inputs are not pooled. All percentages use the observed maximum relative to the observed minimum; they bound only the measured runs, not future executions.

[Partial: observed count allowances; required budget margins remain unmeasured.] These measurements identify where stable accounting was obtained and where a replay policy would have to address observed count variability. They do not show that only tactic-search-dominated mathematics needs such treatment: the selected mathematical stored proofs also have stable counts, their kernel arm has only one active theorem, and search paths were not traced. Shared preparation, realization and allocation attribution remain alternative explanations. Nor does2.33% establish sufficient or necessary `maxHeartbeats` padding: that limit applies per Core context, not to the aggregate measured command deltas, and this experiment does not sweep acceptance thresholds. The later2,524-helper census is separately qualified below and extends the earlier24-target sample within the same three pinned modules and proof form. It does not extend the result to other computational proofs. General computational or mathematical determinism is not claimed.

Orthogonal controls support this bounded interpreta-

tion: 256 cold/warm comparisons preserve verdicts and diagnostics and128 synchronous count vectors match, while100 valid lower-pass/higher-budget pairs across five budgets have no monotonicity counterexample. These are finite observations under their stated cache and adapter settings, not universal replay or monotonicity guarantees.

[**Partial: historical setting controls; general reproducibility unresolved.**] Six released EPFL exact-program controls fail at recursion depth128 but pass at256/default512 in reconstructed contexts; producer context is unknown. A historical `-M8192` diagnostic changes seven error flags among 200 profiler/control pairs. That is neither full-source accepted-set loss nor a current no-`-M` observer-cost estimate. A C-timing control preserves diagnostics but changes allocation-heartbeat deltas; equal error flags do not establish exact budget accounting. [**RESULT: General cross-machine observer/outcome stability and broad software coverage.**]

[**Measured: E0255; finite corrected-adapter matrix.**] A fresh no-`-M` matrix contains 1,536 configuration observations of 16 selected retained miniF2F inputs, with two budgets, stock/synchronous frontends, three observers, four thread counts and reversed-order repeats. Ten corrected qualification controls distinguish valid, false, sorry, parse-error and extra-axiom inputs; they do not establish general transparency. At budget 1,000, off/trace comparison has eight acceptance-flag changes among 256 pairs. All come from one input repeated across configurations, and all occur in the synchronous accounting path: accepted without tracing, budget-exhausted with tracing. The stock/synchronous comparison has eight changes among 384 pairs at that budget; at 3,000 it has none. Thread-count comparisons show no acceptance differences at either budget. Thus this is not a demonstrated trace-induced stock-frontend pass-set loss. It does defeat observer-independent accounting for the tested synchronous path. Allocation observations are partial, not a complete candidate work budget, and these selected observations imply neither population frequency nor cross-machine stability. The 17 native phases and recorded identities retire without resource stops; no new deterministic-budget mechanism follows.

[**Measured: E0256; separate finite budget ladder and input-file control.**] Sixteen selected sources on the synchronous, observer-off path yield 160 observations at five budgets and one/eight available threads. All 100 lower-budget-accepted/larger-budget comparisons retain acceptance. This supports only those chains, not monotonicity for arbitrary sources, observers or machines. Eight intended budget-aware controls fail in the adapter's imported-constant realization before their intended logic runs; they supply no monotonicity counterexample. A separate three-pair input-JSON residency treatment preserves verdicts and diagnostics in 48 comparisons. Only seven request-file pages are evicted or warmed: SDK/Mathlib cache state is not cold. Fifteen parent-thread counts differ by one; asynchronous work is outside that counter, and these observations do not isolate caching as the cause. A later private-runtime cache point now completes this narrower treatment.

[**Measured: E0257; same-machine private-input residency experiment.**] After six runtime qualification controls, eight independent native phases produce 512 configuration rows and 256 matched cold/warm comparisons. Sixteen retained sources use budgets 1,000/3,000, stock/synchronous paths, one/eight available threads and two reversed treatment orders. All comparisons preserve verdicts, diagnostics and axiom-policy results. All 128 synchronous command-counter vectors match; 42 stock parent-thread counter pairs differ, by up to 24 raw units. Those parent counters exclude asynchronous candidate work and do not isolate a causal cache effect.

The treatment uses private SDK/import copies: 101,743 files totaling 8,846,348,681 bytes. Metadata and inode checks distinguish them from live original inputs; no whole-library bytewise comparison is claimed. After treating all selected files and the source/request inputs, residency checks report zero versus all 2,238,400 pages resident. This is cold/warm private input residency, not a cold operating-system boot, global eviction or cross-machine replay. Cold native phases take 66.18–72.51 seconds and warm phases 8.83–15.04 seconds; these are complete harness/import/frontend clocks, not kernel-only speedups. Preparation and file treatment are outside those native clocks. No OOM or sampled-RSS stop occurs; all nine phases and recorded identities retire.

The predecessor fails while compiling the probe because its private executable input closure lacks required IR-signature inputs. It exits 1 after 12.079 seconds, with zero candidate rows; its failure is preserved separately. The corrected point's stability is finite positive evidence for this same-machine treatment, not observer-independent accounting, complete candidate budgets or universal verdict reproducibility.

[**Measured: E0258; completed mathematical subset of a partial co-execution attempt.**] Eight concurrent mathematical batch cells produce 256 rows for the same 16 selected sources and two budgets, with one/two/four/eight recorded worker thread IDs and two opposite execution orders. Worker-one versus higher-worker configurations have no verdict or acceptance-flag changes in 192 comparisons, but 84 command-counter vectors differ. These configurations preserve observed acceptance without establishing exact allocation-count transparency or isolating concurrency as the sole cause.

The planned two-regime comparison stops when its first Envelope process cannot find the target object on its configured import path. It exits 1 after 0.916 seconds, with no Envelope proof-replay rows: an input-availability failure, not original-kernel rejection. The

Stage / regime	Rows	Pairs	Changes
Kernel, computational	192	144	0
Kernel, mathematical	16	12	0
Source, mathematical	128	96	49

Table 20: Finite allocation-count observations, not independent candidate samples or native kernel poll counts. Pairs compare requested worker-one against higher-worker configurations within each order. Mathematical stored-proof cells have only one active checking thread.

four completed Grid prefix cells do not complete the planned repeated computational comparison. The originally planned repeated cross-regime contrast remains incomplete; the later, separately qualified stage panel below does not complete that earlier protocol. No throughput or general determinism claim follows.

[**Measured: E0259; reanalysis of the same partial mathematical subset.**] Of its 192 comparisons, 83 have changed theorem-command allocation counts, 26 have changed opening-command counts, and none has a changed terminal count; these sets overlap. Three pairs also change the nested timeout diagnostic location, while retaining budget-exhausted outcomes. Thus outcome stability does not imply identical diagnostics, and the variation is not confined to opening commands. These records do not identify a changed search tree or isolate allocation's causal source.

[**Measured: E0259; separate, completed original-module stage panel.**] A new qualified capture checks five retained original modules: three computational modules (UMoments9, BarrierData41 and EnvelopePsiB) and two mathematical modules (InteriorH1Derivative and ReindexSingle). Its 58 successful native phases include two stock-source controls and 56 stage cells, with requested worker pools of one/two/four/eight and two reversed orders. Table 20 separates stages rather than treating allocation counts as a complete candidate budget. Every recorded stage comparison preserves verdict and diagnostics.

The kernel cells use eight closed stored theorem proofs per computational module and the sole eligible theorem in each mathematical module. They rename each proof and call the original C++ kernel in the fully imported target context. Computational cells use up to eight recorded checking threads; mathematical kernel cells have one active thread even when a larger pool is requested. Those mathematical cells therefore do not establish concurrent kernel stability. The source cells use eight full-source clones per cell of each of the two mathematical programs, with synchronous elaboration, heartbeat budget 2,000,000 and recursion depth20,000. They are repeated configurations of two programs, not 128 independent candidates or stock-budget equivalents. Their actual active thread counts are one/two/four/eight.

[**Partial: stage-qualified accounting; broader regime and budget claims remain open.**] The observed distinction concerns the measured stage: both regimes' selected stored proofs have stable counts, while the mathematical source commands vary. It does not establish that only mathematical workloads need reproducible accounting. Kernel imports remain trusted; computational full-source cells are absent and native kernel poll counts are unmeasured. Source command clocks include tactics and kernel work but exclude parsing/import/header preparation; subtracting them from the selected-proof clocks would not identify a source-phase share. The capture is no-`-M`, P0–7, with a14GiB charged cap and separate RSS sampling; all117 recorded process identities and cgroups retire. Earlier namespace, linking and eligibility failures remain archived; they contain no completed candidate matrix. [**RESULT: Matched concurrent mathematical-kernel treatment, computational full-source accounting, independent full-closure checking and general cross-machine/global-budget guarantees.**]

[**Measured: E0260; independent three-repeat stage capture.**] A later completed capture repeats the same selected-unit protocol three times at each requested pool size, separately from the earlier two-repeat point. All86 native phases exit0, including two stock-source controls. Its312 stored-proof rows yield234 comparisons with no allocation-count, verdict or diagnostic changes. Its192 mathematical source-clone rows yield 144 comparisons,78 with changed allocation vectors (40/38 across the two programs), and no verdict or diagnostic changes. The mathematical kernel arms still have just one active proof each. Across96 observations per mathematical source, the pooled raw-count ranges are0.036739% and0.138453% above the observed minimum. These finite ranges are not a demonstrated minimum budget margin: the high-limit module experiment does not sweep acceptance thresholds. The same synchronous options, trusted-import boundary, two actual source programs and sampled eight computational helpers per module still apply. Native kernel poll counts and complete candidate work remain unmeasured; the all-helper census below is a separate capture; these earlier three-repeat denominators remain unchanged. All173 recorded native/controller identities and cgroups retire; the earlier capture's denominator is not relabeled as three repeats.

[**Measured: E0262; completed census of eligible computation helpers in three pinned modules.**] The independent census covers all1,164,1,320 and40 local theorem helpers whose annotation-cleaned proof head is `of_decide_eq_true`, in UMoments9, BarrierData41 and EnvelopePsiB, respectively. All36 native processes finish successfully. Every helper appears once per cell, across four worker counts and three fixed-order repeats:30,288 accepted checks of2,524 unique targets. All 22,716 worker-one versus higher-worker com-

parisons preserve allocation counts, verdicts and diagnostics; each target's range across all12 observations is zero. Across targets, both the nearest-rank P95 and maximum relative range are 0%; all 2,524/2,524 targets have zero observed variation. Recorded checking-thread counts match the requested one/two/four/eight workers in every cell.

This is a census of the declared local proof form in these three modules, not of all declarations, source computations or computational proofs. The probe prepares closed stored proofs before accounting, checks them under fresh names with the original kernel, and trusts the fully imported target context. The three target objects match the retained original producer artifacts byte-for-byte. Import/preparation work, transitive axiom closure and native kernel poll counts are outside the allocation observation. Summed checking-thread CPU still varies: its three-repeat median rises from73.958s at one worker to95.416s at eight workers for UMoments9. Equal allocation counts therefore do not establish zero runtime overhead or a sufficient acceptance-preserving budget margin. All73 recorded process identities and36 cgroups retire without resource stops. This qualified census strengthens the finite applicability evidence while leaving cross-machine and complete-candidate budget claims unresolved.

The retained candidate contrasts have different failure/cost boundaries. One miniF2F point spends73.21% of summed physical worker-attempt time on budget-source attempts, not that share of concurrent makespan. ProofNet pass1 has an11.57% budget-source request-time share. AutoVerus's slowest1.6% consumes about3.4%, a slow-tail rather than budget-failure metric. These historical resource points aren't a regime classifier for the library. Ordinary errors and resource exhaustion do not establish falsehood.

Parallel checking preserves observed counts and serialized payloads. [Measured: E0267; finite streaming and serialization comparison.] A separate v16 process checks Barrier's 4,448-module dependency prefix, then repeats its entire 3,425-declaration leaf in six waves at per-wave worker caps 1, 4, 8 and 16, three times each, reversing order in the middle repeat. The 100 observed helpers are evenly spaced in the sorted eligible list and overlap the earlier Barrier census; they are not additional independent targets or a new population sample. All 1,200 observed checks accept, and every target has zero allocation-count range across its 12 observations: both P95 and maximum are 0%. All leaf constants structurally match the one-worker reference. Each trial substitutes all 3,425 checked constants into the module metadata and actually serializes an output: all twelve 9,160,440-byte `.olean` files match one another and the original input byte for byte. Only that one artifact part exists for this input; no absent companion is counted as equal. Extension metadata retains the input policy and is not independently revalidated. This establishes neither whole elaborator/axiom equivalence nor byte identity of every transitive artifact.

These observer-enabled trials share a resident checked prefix and the allowed 8P+8E mask; workers may migrate within it. Median leaf wall decreases from 50.358 to 9.033 seconds at caps one and 16, while leaf process CPU rises from 50.334 to 139.099 seconds. Those windows include leaf planning/checking/structural audit and observer work, but exclude the following serialization; they are not unprofiled complete-process throughput. CPU summed inside the same 100 executing-worker call windows rises from 3.575 to 9.941 seconds despite fixed allocation counts. The clocks include counter reads around the original-kernel call and precede per-row logging; no cause among placement, frequency, contention and other runtime effects is isolated. Distinct TIDs accumulated over six waves are unions over time, not simultaneous concurrency counts. The complete native process takes 556.858 seconds and 1,651.508 CPU seconds, including prefix checking, all twelve trials and all serialization. This finite result strengthens the prepared-stage count and output boundary, without establishing zero observer/runtime cost, native kernel poll counts, a near-threshold budget guarantee or a future-machine property. It is separate from both the earlier 2,524-target protocol and the negative heterogeneous complete-closure curve.

[Measured: E0273; separate resident hardware calibration.] One further process checks the complete Barrier closure, then runs nine resident leaf trials: P1 on CPU0, E1 on CPU8 and mixed24, three repeats each with the middle order reversed. Fifty overlapping real helpers yield 450 accepted observations; every target's allocation-count range is zero. All nine actually emitted 9,160,440-byte `.olean` files equal the original input byte for byte, with 3,425 checked payload substitutions and trusted extension metadata. These helpers are not added to the earlier 2,524-target census. Normalizing each mixed check against that target's median solo CPU cost on the same core type gives median ratios 2.301 for P and 2.834 for E; all 150 mixed observations cost more CPU than their solo-type reference. The references are CPU0 and CPU8, not the same physical core or controlled frequency for every mixed observation. This shows concurrency-associated CPU growth at fixed observed counts, without identifying an instruction-level cause or a general budget guarantee. The compound native process costs 782.484 wall and 1,688.835 CPU seconds, including the checked prefix, all nine trials and serialization; it is not nine independent complete-closure measurements.

System-design implication. [Partial: implication of scoped observations; general benefit unproven.] Report build domain, checked constants, trusted imports, axiom/admission policy, exe-

cutable/options, resource accounting and observer configuration. A Boolean result hides these boundaries. Faster delivery of a narrower checking scope isn't equivalent verification.

7 Limits and unfinished evidence

The complete lexical comparison establishes concentration, not runtime prevalence or a universal two-way partition. The no-`-M` sample remains incomplete and purposeful modules receive no population weights. **[RESULT: Current per-regime profiles, expansion estimates and uncertainty.]** Generated-software contexts/provenance differ; software-stage costs remain **[RESULT: pending]**.

The prototypes have bounded acceptance and cost observations, rather than general accepted-set transparency, soundness or cluster results. Broader differentials, axiom closure, failures and resource-qualified lifecycle comparisons remain open. Ordinary warm-state reuse and frozen arithmetic oracles are prior mechanisms, not the contribution. The historical GPU screen was slower and removed no timeout; that does not rule out all devices or workloads. All failed, partial and withdrawn evidence remains, including interrupted state-reuse records whose missing entries aren't zero cache opportunity.

8 Related work

Closest same-corpus cost study. A public technical report already studies another revision of `openai/math`, including build/check costs, source-based forecasts, computation-heavy checking outliers, parallel external checking, default-root coverage and verification meanings [32]. Its machine load and timing/acceptance boundaries differ from ours; we do not transfer its numbers or dismiss the conceptual overlap. We claim neither the first cost characterization of this corpus nor the first observation that checking may recompute. The empirical increment examined here is controlled boundary matching, explicit missingness and tested limitations of proposed responses.

Parallel proof checking and deterministic budgets. Wenzel's Isabelle proof futures check promised propositions and well-founded fulfilment dependencies [46]. Coq's asynchronous document processing separates opaque proof tasks and complete checking [2]; its scheduler already predicts task durations using historical auxiliary data [6]. Lean 4.17 overlaps kernel checking with elaboration, and 4.19 parallelizes theorem-body elaboration [15, 14]. Certificate splitting and composition also have established precedents [42]. These are citation precedents, not performance baselines we have rerun. Likewise Lean's thread-local allocation heartbeats and SMT-LIB's reproducible-resource option precede candidate B [24, 3]. Neither their existence nor our controls establish a new whole-pipeline replay guarantee. SATzilla already uses instance predictions for algorithm decisions [48]; candidate A requires application-specific evidence.

Proof-assistant runtime and build performance. Performance engineering of proof assistants predates AI-generated proofs. Lean's architecture and reference-counting work study efficient execution of functional metaprograms [7, 44]. Tebbi and Gross's Ltac profiler attributes nested tactic invocations in Coq [43]. Huch and Wenzel study distributed Isabelle/AFP builds and schedules for multithreaded sessions [10]. These establish runtime, tactic and dependency-scheduling costs as important problems. Our work measures which of these boundaries dominate particular generated artifacts; low realized core use does not by itself establish a new scheduling or runtime technique.

Mathlib compilation and continuous integration. The Mathlib maintenance study already describes continuous build/component performance tracking and optimization [1]. The official Mathlib benchmark suite already records global and per-module build metrics, including task time, wall time, instructions, RSS and profiles [29]. Standard stratification and known inclusion probabilities are established sampling methods [40]; our method applies them to this frame, with incomplete response kept explicit. Community profiling has already examined per-file build times and typeclass, tactic, import and checking phases in Mathlib [41]. We do not claim the first library cost profile. Mathlib's official cache downloads compiled artifacts keyed by toolchain, source and imported dependencies; CI constructs and publishes those artifacts [28]. This makes incremental CI, cached downstream builds and fresh source compilation different cost boundaries. Prior Isabelle-build work also discusses Mathlib's cached workflow in its contemporary setting [10]. We do not import its CI duration or community profiler percentages into our hardware/version denominators. Our new-source study distinguishes default-root coverage, successful artifact growth, CPU/system time and uncompleted probability draws rather than treating a cache hit or a build command's completion as all-source coverage.

Native replay and external proof checkers. The archived `lean4checker` is now distributed with Lean as `leanchecker`. It replays declarations through the official kernel: it is a separate checking pass, not a separately implemented kernel [25]. Nanoda independently implements checking in Rust and exposes checking workers and explicit axiom/extension settings [31]. Carneiro's

Lean4Lean reimplements the checker in Lean and evaluates post-compilation replay on major libraries [5]. Lean Kernel Arena evaluates multiple implementations on exported environments, including acceptance, rejection and corner cases [23]. Its instruction-derived virtual CPU scores and a local compiler wall clock have different meanings. These studies address checking performance and trust at post-export or compiled-environment boundaries. Our measurements additionally charge source elaboration, imports, emitted artifacts and failed candidate work; a faster external kernel is useful only for the share at its boundary. Axioms, admissions and native evaluation remain separate trust questions.

RL verification services and prepared proof state. Kimina Lean Server uses concurrent persistent REPLs and cached import headers for RL and proof feedback [34, 36]. Its published throughput study establishes import reuse as a practical service technique, not a new mechanism here. DeepSeek-Prover-V1.5's verifier already provides process scheduling, REPL checks, timeouts and distinct pass/complete/system-error fields [8]. These services are source-feedback paths, not matched independent-artifact replay baselines. Snapshots retain live proof state for tactic search [38]; stock incremental elaboration reuses prior source state [18]. APE-Bench evaluates file/repository-level proof engineering with pinned library versions and shared infrastructure [47]. Such services and benchmarks establish why reusable contexts matter. Our comparison asks where resource-bounded time and memory go across research mathematics, software artifacts and candidate streams, and whether profiling or changed limits alter their measured verdicts. We do not transfer service speedups to a serial library build.

Reuse, scheduling and certified computation. Fine-grained verification caching [26], tactic/prefix reuse [4, 39], simplifier memoization [16], dynamic dependencies [35, 9] and age-based scheduling [27, 37] are established alternatives. The streaming and chunking prototypes test different regime-specific implications; external search with native checking is prior art.

The empirical increment examined here is a boundary-consistent, two-regime study of released AI-generated formal artifacts, with a research-level library as the primary population and rejected/resource-limited attempts retained as contrasts. It joins source/build coverage to explicitly scoped native costs, treats mapped resident pages and mutable state as distinct metrics, and tests whether observation preserves outcomes. It complements build engineering and checker benchmarks rather than claiming that their bottlenecks are new. Complete software and causal-profile findings remain explicit open results.

9 Conclusion

[**Measured:** **E0231**,**E0235**] The fixed release has concentrated kernel-decision syntax and selected cheap-witness versus costly-recomputation cases at declared cost boundaries. [**Partial: population runtime weights, causality and generality missing.**] A has bounded negative/tradeoff observations but no per-instance bound or completion gain in the source-compilation point. Historically calibrated artifact admission improves known-panel availability without unseen-root safety; B remains unproven as a general guarantee. C is a measured finite-index application of established decomposition and checking, with additional implementation trust and incomplete full costs. [**RESULT: Final no-`-M` regime weights, matched native module-task comparison, complete Envelope single-mode pair, software/axiom and cluster evidence.**] The defensible contribution is the scoped characterization and tested implications, not three general new mechanisms.

References

[1] Anne Baanen, Matthew Robert Ballard, Johan Commelin, Bryan Gin ge Chen, Michael Rothgang, and Damiano Testa. Growing mathlib: maintenance of a large scale mathematical library, 2025. Section 5 inspected; existing build/component/instruction performance studies.

[2] Bruno Barras, Carst Tankink, and Enrico Tassi. Asynchronous processing of coq documents: from the kernel up to the user interface, 2015.

[3] Clark Barrett, Pascal Fontaine, and Cesare Tinelli. *The SMT-LIB Standard, Version 2.7*, 2025. Section 4.1.7, reproducible-resource-limit; scope includes machine, command history and random seed.

[4] Lasse Blaauwbroek, Josef Urban, and Herman Geuvers. The tactician (extended version): A seamless, interactive tactic learner and prover for coq, 2020. Extended version of the CICM paper.

[5] Mario Carneiro. Lean4lean: Verifying a typechecker for lean, in lean, 2025. Version 3, revised 14 September 2025; kernel replay, not source-compilation timing.

[6] Coq contributors. Asynchronous and parallel proof processing, 2023. Version 8.17.1; auxiliary-file duration prediction and proof-task checking.

[7] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[8] DeepSeek-AI. Deepseek-prover-v1.5: Lean verification artifact, 2024. Current official artifact inspected 1 November 2026, not a remeasurement of the publication.

[9] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. Adapton: Composable, demand-driven incremental computation. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 156–166, 2014.

[10] Fabian Huch and Makarius Wenzel. Distributed parallel build for the isabelle archive of formal proofs. In *15th International Conference on Interactive Theorem Proving (ITP 2024)*, volume 309 of *LIPIcs*, pages 22:1–22:19, 2024.

[11] Lean community. Mathlib, 2026. Accessed 2026-10-07; year denotes access year.

[12] Lean community. Mathlib4 reference source at d13f23b7. https://github.com/leanprover-community/mathlib4/tree/d13f23b723b8a846827a245b89c10fc7d3f11612/Mathlib, 2026. Pinned Mathlib subtree; local Git revision and unchanged source checked 1 November 2026. Lexical census excludes comments and strings.

[13] Lean contributors. Lean 4.9.0-rc1 ordinary decision tactic implementation. Pinned evalDecide handler; accessed 30 October 2026.

[14] Lean contributors. Lean 4.17.0: Parallel kernel checking, 2025. Kernel checking concurrent with elaboration.

[15] Lean contributors. Lean 4.19.0: Parallel elaboration, 2025. Theorem-body parallel elaboration; source-release precedent.

[16] Lean contributors. Lean4.22.0 release: simplifier cache reuse, 2025. Simp section; PR 8880.

[17] Lean contributors. Do not count evaluating shared thunks toward heartbeats, 2026. Upstream commit inspected; backport to the measured 4.34.1 SDK unestablished.

[18] Lean contributors. Lean 4.32.0 release: incremental snapshots, 2026. Accessed 2026-10-07; year denotes access year.

[19] Lean contributors. Lean 4.34.1 auxiliary-lemma implementation, 2026. Branch-local cache lookup; accessed 30 October 2026.

[20] Lean contributors. Lean 4.34.1 bundled checker: module task dispatch. https://github.com/leanprover/lean4/blob/v4.34.1/src/LeanChecker.lean, 2026. Pinned SDK source inspected 1 November 2026; independently dispatched module tasks, not a measured whole-library baseline.

[21] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026. doKernel invokes mkAuxLemma; accessed 30 October 2026.

[22] Lean contributors. Lean 4.34.1 runtime memory-budget checks, 2026. Source and matching SDK disassembly inspected 31 October 2026.

[23] Lean contributors. Lean kernel arena, 2026. Accessed 2026-10-07; year denotes access year.

[24] Lean contributors. Lean language reference: Io timing, 2026. Thread-local small-allocation heartbeats; no universal cross-condition guarantee.

[25] Lean contributors. lean4checker and the built-in leanchecker, 2026. Official archived README; integrated into Lean from v4.28.0; accessed 31 October 2026.

[26] K. Rustan M. Leino and Valentin Wüstholz. Fine-grained caching of verification results. In *Computer Aided Verification (CAV), Part I*, pages 380–397, 2015.

[27] Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal speedup of Las Vegas algorithms. *Information Processing Letters*, 47(4):173–180, 1993.

[28] Mathlib contributors. Mathlib build cache and ci uploads, 2026. Official cache README and Cache/CI.md, accessed 31 October 2026; general CI context, not measured CI cost.

[29] Mathlib contributors. Mathlib4 benchmark suite: build metrics, 2026. Official README and run script inspected; not current numerical CI costs.

[30] Meta Platforms. Lean Kernel Verify: Compiler-based safety checker for gpu kernels, 2026. Pinned release 129e65bfd8efbf6c45710732d205c941ef65928c; accessed 30 October 2026.

[31] Nanoda contributors. Nanoda: external lean type checker, 2026. Current official README inspected; workers, axioms and kernel extensions are configurable; no local benchmark.

[32] nealmcb. Verifying openai/math in lean: cost and checker study, 2026. Public technical report; pinned report revision, sections 1b, 3c, 3f, 4 and 5 inspected; workload revision and measurement conditions differ.

[33] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[34] Project Numina. Kimina lean server, 2026. Accessed 2026-10-07; year denotes access year.

[35] Salsa contributors. The red-green algorithm, 2026. Official algorithm documentation; accessed 30 October 2026.

[36] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[37] Ziv Scully, Mor Harchol-Balter, and Alan Scheller-Wolf. SOAP: One clean analysis of all age-based scheduling policies. arXiv:1712.00790v2, 2018.

[38] Austin Shen and Yunong Shi. Keep the proof state live: Snapshotting for efficient tactic search in lean 4, 2026.

[39] SHRED contributors. SHRED: Reuse repeated work across thousands of Lean proof attempts, 2026. Software and experimental artifact; revision a6616e99534ce777dcebfd56566eaaffb63997a8; accessed 16 October 2026.

[40] Statistics Canada. Statistics: Power from data! probability sampling. Accessed 1 November 2026; standard sampling precedent.

[41] Michael Stoll. Ways to speed up mathlib. Lean community Zulip archive, 25 April 2024, 2024. Community measurement report; no timing or phase percentage transferred to our experiments.

[42] Pierre-Yves Strub, Nikhil Swamy, Cédric Fournet, and Juan Chen. Self-certification: Bootstrapping certified typecheckers in f* with coq. In *POPL*, pages 571–584, 2012. Section 7: source-level certificate splitting and composition.

[43] Tobias Tebbi and Jason Gross. A profiler for ltac. CoqPL Workshop abstract, 2015.

[44] Sebastian Ullrich and Leonardo de Moura. Counting immutable beans: Reference counting optimized for purely functional programming. In *Proceedings of the International Symposium on Implementation and Application of Functional Languages (IFL '19)*, 2020.

[45] Vericoding contributors. Vericoding: released model-tagged programs and final experiment tables. Artifact repository, revision b265a3b2bce31e09d264306f419e810d02f98f44, 2026. Inspected 31 October 2026; final outputs do not constitute complete verification trajectories.

[46] Makarius Wenzel. Parallel proof checking in isabelle/isar. In *ACM SIGSAM Workshop on Programming Languages for Mechanized Mathematics Systems (PLMMS)*, 2009. Author PDF, sections 3–4 inspected via CLI; original-host web fetch unavailable.

[47] Huajian Xin, Luming Li, Xiaoran Jin, Jacques Fleuriot, and Wenda Li. Ape-bench: Evaluating automated proof engineering for formal math libraries, 2026. Version 3, revised 2 February 2026.

[48] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.

[49] Lin Xu, Frank Hutter, Jonathan Shen, Holger H. Hoos, and Kevin Leyton-Brown. Satzilla2012: Improved algorithm selection based on cost-sensitive classification models, 2012. Solver description; sections I–II inspected; classification without runtime regression is prior art.

[50] Zhe Ye, Hantao Lou, Yuechun Sun, et al. Vero: Can ai agents build formally verified software repositories?, 2026. Official repository inspected at 0a7325df9e9e6dbc275c0ad483b3d1cbe38d9b09; evaluated run outputs not established.